AI · ANALYTICS · MUSIC INFORMATION RETRIEVAL

seekaudio

Complete technical reference and personal learning guide

A detailed explanation of the implemented audio-analysis and content-based music-retrieval system: theory, formulas, code flow, evaluation evidence, and a from-scratch study path.

Implementation snapshot: 12 October 2026
Prepared from the project's annotated Python code and saved version 2 results.

Part I — Technical report: 24 chapters

Part II — Learning guide: 12 progressive stages

Part III — Method reference: 96 functions/classes

Catalog retrieval scores are not held-out genre-classification accuracy. Future research ideas are distinguished from existing functionality.

PART I — TECHNICAL REPORT

seekaudio: technical reference report

Implementation snapshot: 12 October 2026.
Scope: the current Python implementation, saved version 2 catalog, Streamlit analysis interface, and learning context.
Companion documents: Learning guide and Method reference.
Audience: a developer returning to Python and preparing for further study in AI, analytics, and audio.

This is a reference document, not a claim of a new state-of-the-art algorithm. Implementation statements come from the repository's actual methods. Numerical results come from the saved evaluation artifacts. Background references are linked where useful; optional future work is explicitly distinguished from implemented behavior.

Contents

Purpose and research question

MIR context and task boundaries

Architecture and processing flows

Dataset, metadata, and artifacts

Notation and default settings

Digital audio preparation

Frames, windows, and Fourier analysis

Pooling and fixed-size representations

RMS energy

Spectral descriptors

Mel filters and MFCCs

Stored rhythm features

Optional tonal features

Librosa display analysis

Search-space construction

Cosine retrieval and genre voting

Mel spectrogram and displayed measurements

Evaluation definitions and worked example

Recorded results and interpretation

Implementation, configuration, and deployment

Tests, complexity, and reproducibility

Limitations and next research steps

Glossary and misconceptions

References and evidence

---

1. Purpose and research question

seekaudio analyzes a recording's measurable sound properties and uses those properties to compare it with a catalog. It has two related interfaces:

A CLI retrieves acoustically similar catalog recordings.

A Streamlit page accepts an uploaded song or bundled sample, estimates genre by voting over retrieved neighbors, and displays technical audio characteristics and a Mel spectrogram.

A suitable research question is:

> How do interpretable timbral, spectral, energy, and rhythm descriptors contribute to music retrieval under a same-genre relevance proxy?

The project examines representations, not just whether a UI can display a plausible label. The eight ablations are a controlled way to compare feature choices.

The application does not learn from listening history, user profiles, artist metadata, or song popularity. It does not identify an exact recording by an audio fingerprint, transcribe lyrics, separate instruments, or train a neural audio embedding. Its genre output is a nearest-neighbor decision rule over a fixed catalog.

There is still machine learning: the scaler and PCA are fitted to catalog data. PCA is unsupervised. Genre labels enter evaluation and the final voting rule, not the acoustic feature calculations or PCA fit.

2. MIR context and task boundaries

Music Information Retrieval (MIR) studies computational ways to analyze, organize, search, and interact with music. Music can be represented as audio, scores, symbolic events, metadata, or combinations of these. Audio retrieval by example is one MIR task: supply a recording and rank related recordings. The FMP materials provide a broader map of MIR topics and algorithms.

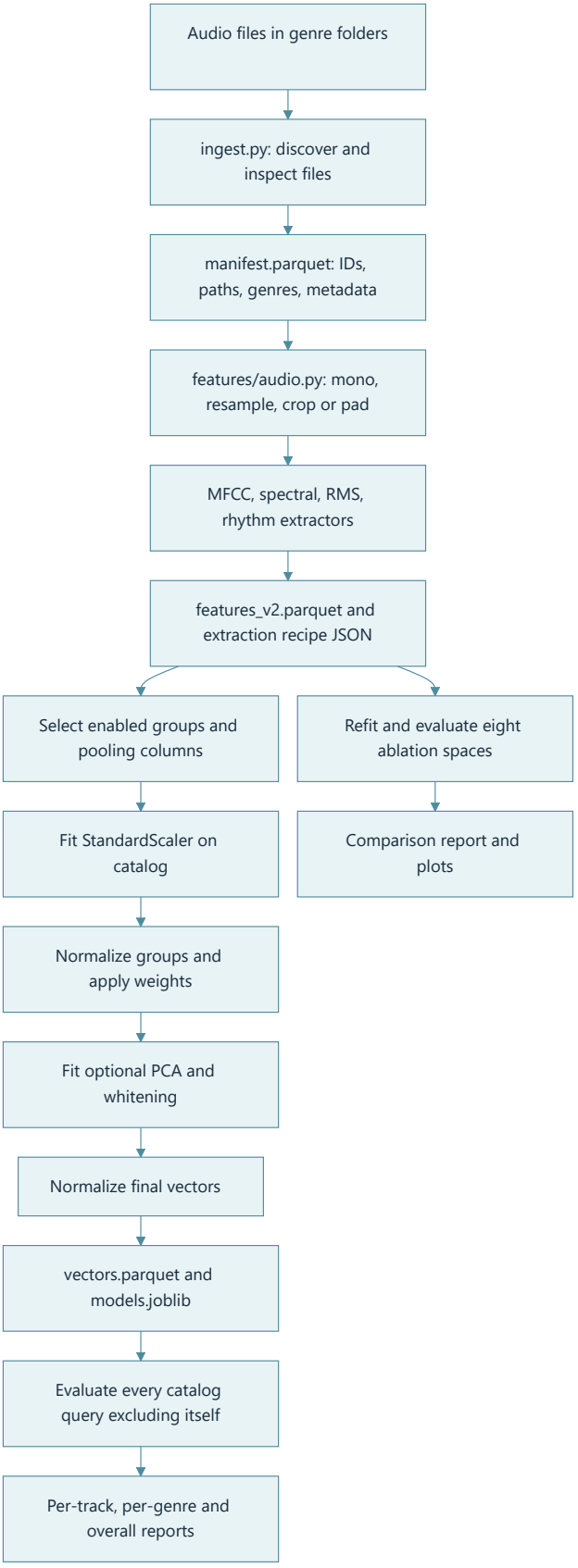
Task	Question	Relationship to seekaudio
Acoustic content-based retrieval	Which recordings have similar measured sound?	Implemented
Genre estimation	Which catalog genre is most common among neighbors?	Implemented as a vote
Audio fingerprinting	Is this the same recording, including a short excerpt?	Not implemented
Cover/version identification	Is this another performance of the same composition?	Not implemented
Beat tracking	At which times do beats occur?	Not implemented; tempo is estimated
Music transcription	Which notes were played, and when?	Not implemented
Recommendation	What should this particular listener hear next?	No personalized recommendation model
Source separation	Which samples belong to vocals, bass, drums, etc.?	Not implemented

Timbre concerns the sound character distinguishing sources with comparable pitch and level. MFCCs describe aspects of the spectral envelope; they do not directly label individual instruments.

Musical similarity is not one universally defined quantity. Two songs can share tempo but differ in instrumentation, share harmony but differ in production, or belong to different genres despite similar spectra. A cosine score in this project means similarity in its chosen representation.

3. Architecture and processing flows

3.1 Offline MIR catalog preparation

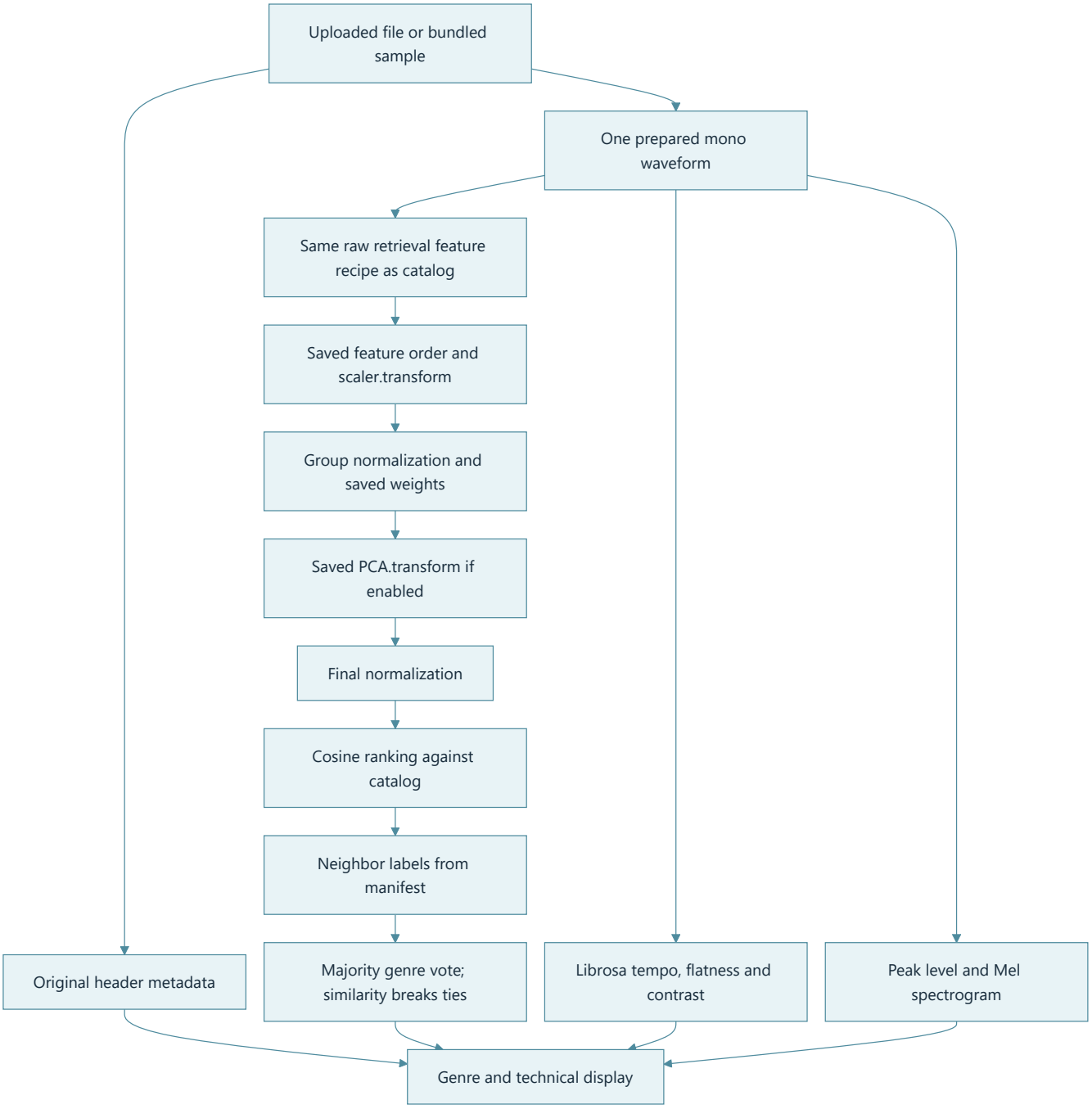


Entry point: `python -m src.seekaudio run`.

Coordinator: `pipeline.run_pipeline()`.

Important: the catalog is fitted during preparation, not whenever someone uploads audio.

3.2 Live MIR analysis flow



Coordinator: `analysis.analyze_audio()`.

UI entry point: `streamlit_app.main()`.

The live path never calls `scaler.fit` or `pca.fit`. An uploaded query must enter the coordinate system learned from the catalog.

Librosa's additional display measurements branch off from the waveform. They do **not** change the saved 53-dimensional raw retrieval schema or catalog evaluation scores.

3.3 Representation sizes

Object	Default shape	Meaning
Original decoded stereo audio	`(original_samples, 2)`	Decoder layout before mono conversion
Prepared mono waveform	`(661500,)`	30 seconds at 22,050 Hz
Complete audio frames	`(1288, 2048)`	Frame-first layout
Real FFT magnitude/power	`(1288, 1025)`	Non-negative frequency bins
Custom Mel power	`(1288, 128)`	128 Mel bands per frame
Selected MFCC matrix	`(20, 1288)`	Coefficient-first layout after transpose
Raw catalog feature table	`(999, 54)`	53 numbers plus ID
Default PCA vector table	`(999, 41)`	40 numbers plus ID
Librosa input spectrum	`(1025, 1288)`	Frequency-first layout
Display Mel matrix	`(128, 1288)`	Mel band versus time

Numbers apply to the current defaults. They change with window sizes, duration, coefficient counts, group selection, or optional tonal features.

---

4. Dataset, metadata, and artifacts

4.1 Local catalog

The local GTZAN copy contains 1,000 discovered recordings arranged in ten genre folders. One file, `jazz.00054.wav`, cannot be decoded by SoundFile and is skipped with a warning. The saved manifest therefore has 999 usable entries: nine genres with 100 tracks each and jazz with 99.

GTZAN is useful for learning and baseline experiments, but its known repetitions, labeling issues, and recording artifacts require care. The dataset analysis repository maintained by Bob Sturm documents these concerns. This project has not performed a complete duplicate/artist/content audit.

4.2 Manifest fields

Column	Definition	Example
`track_id`	Filename without extension	`blues.00000`

Column	Definition	Example
`path`	File path stored during ingestion	`data/raw/.../blues.00000.wav`
`genre`	Name of immediate parent directory	`blues`
`duration`	Original recording duration, seconds	About 30
`sample_rate`	Original samples per second, Hz	22050
`channels`	Original channel count	1

The genre is inferred from directory structure, not recognized from the header. IDs must be unique for search. Copies with the same stem in different folders can violate this assumption.

Metadata is not an MFCC, tempo, or spectral measurement. It describes the file's organization and technical properties.

4.3 Saved artifacts

Artifact	Purpose
`data/manifest.parquet`	Catalog identity, genre labels and metadata
`data/features/features_v2.parquet`	Raw acoustic values
`data/features/features_v2.json`	Exact extraction settings and feature version
`data/space/vectors.parquet`	Ready-to-search catalog representation
`data/space/models.joblib`	Fitted scaler/PCA and deterministic transformation settings
`data/evaluation/current/`	Current catalog metrics and configuration
`data/evaluation/ablations/`	Scores and configurations for eight experiments
`data/evaluation/plots/`	Genre and ablation figures
`data/demo/sweet_child_o_mine.mp3`	Optional locally supplied sample
`deployment/streamlit/`	Generated small runtime bundle

Parquet is a typed columnar table format. Joblib serializes Python model objects. JSON records readable settings. A model's column order and extraction recipe are as important as its fitted numeric parameters.

The web interface needs the manifest, vectors, models, source, dependencies, and configuration. It does not need raw GTZAN recordings because it no longer plays retrieved catalog tracks. Manifest paths can describe local Windows files without those paths being opened by the live analysis-only interface.

5. Notation and default settings

Symbol	Meaning
$x[n]$	Prepared mono sample at index n
f_s	Sample rate in Hz
L	Number of prepared samples
N	Samples per frame; `n_fft`
H	Hop length between frame starts
T	Number of complete frames
$F = N/2 + 1$	Real FFT bins for even N
M	Number of Mel bands
C	Retained MFCC coefficient count
N_c	Number of catalog tracks
D	Number of raw selected feature columns
P	PCA component count
K	Number of retrieved neighbors
$A_{t,k}$	Spectral magnitude in frame t, bin k
$P_{t,k}$	Spectral power, $A_{t,k}^2$
$\mathbf{v}_i$	Final representation of catalog track i

Defaults: $f_s = 22050$, duration 30 seconds, $N = 2048, H = 512, M = 128, C = 20, D = 53$, $P = 40$, and $K = 10$.

`feature_spec_version=2` identifies the stored retrieval recipe. Adding display-only Librosa values does not change its raw columns. Changing an extractor definition would require a new version and rebuilding compatible artifacts.

6. Digital audio preparation

6.1 Sampling and channels

Continuous sound pressure is represented digitally by samples:

$$x[n] \approx x_{\text{continuous}}(n/f_s).$$

Hz means cycles per second for frequency and samples per second for sample rate. These are related units but different concepts.

For C_h decoded channels, mono conversion is an arithmetic mean:

$$x_{\text{mono}}[n] = \frac{1}{C_h} \sum_{c=1}^{C_h} x_c[n].$$

This simplifies downstream shapes and computation. Opposite-phase stereo content can cancel; mono conversion does not preserve every spatial property. The UI still reports the original file's channel count.

6.2 Resampling

For source rate f_o and target f_s:

$$g = \gcd(f_o, f_s), \quad u = f_s/g, \quad d = f_o/g.$$

`resample_poly` performs filtered rational resampling by factors u, d. Filtering suppresses frequencies incompatible with the lower target rate; simply taking every other sample is not an adequate general resampler. See the SciPy API for implementation parameters.

For 48,000 to 22,050 Hz, $g = 150, u = 147, d = 320.$

Nyquist frequency is $f_s/2 = 11025$ Hz for the prepared audio. Frequencies above that cannot be represented uniquely after downsampling. **Aliasing** is the folding of incompatible high-frequency content into lower frequencies.

The app is not measuring the full original 48 kHz spectrum: its analyzed version uses 22,050 Hz.

6.3 Centered excerpt and padding

Target length:

$$L = \text{round}(f_s \, d_{\text{seconds}}) = 661500.$$

If the decoded/resampled waveform has length $L_o > L$:

$$s = \left\lfloor \frac{L_o - L}{2} \right\rfloor, \qquad x_{\text{excerpt}} = x[s : s + L].$$

If $L_o < L$, let $m = L - L_o$. Left padding is $\lfloor m/2 \rfloor$; right padding is the remainder. Samples outside the recording are zero.

The displayed duration describes the complete original file. Its measurements and spectrogram describe the centered, padded-if-needed excerpt.

Padding is convenient but not neutral: it can lower pooled energy, affect feature statistics, and create artificial silence regions. Mean/std pooling does not remove this effect.

6.4 Sample type and level

The loader returns `float32`. Some later arithmetic uses `float64` for stability. These are storage/precision choices, not bit-depth labels of the original recording.

Values near $[-1, 1]$ are common for normalized decoded PCM, but the code does not apply peak normalization or a loudness normalization standard. Floating-point input can exceed one.

Prepared sample peak:

$$a_{\max} = \max_n |x[n]|, \quad L_{\text{peak}} = 20 \log_{10}(a_{\max}) \text{ dBFS}.$$

For a zero waveform, the UI shows silence rather than trying to display $\log 0$. Zero dBFS means unit sample amplitude; it does not by itself prove audible clipping. This is not inter-sample true peak, sound pressure level, or LUFS.

---

7. Frames, windows, and Fourier analysis

7.1 Why use short frames?

A song changes over time. A single FFT of the complete recording loses the local context of attacks, notes, and changing instrumentation. The project computes spectra over overlapping windows, then summarizes their measurements.

Frame t:

$$x_t[n] = x[tH + n], \quad 0 \le n < N.$$

Only complete frames are used:

$$T = 1 + \left\lfloor \frac{L - N}{H} \right\rfloor = 1288.$$

The final 508 samples are not part of a complete frame under these defaults. The actual last frame ends at sample 660,992 (exclusive), rather than using every sample of the 661,500-sample excerpt.

Frame duration: $N/f_s \approx 92.88$ ms.
Hop duration: $H/f_s \approx 23.22$ ms.
Frame rate: $f_s/H \approx 43.07$ frames/second.

A view created by `sliding_window_view` avoids copying every raw overlapping window. Later window multiplication and spectral arrays still allocate memory.

7.2 Hann window

The exact symmetric window used by `np.hanning(N)` is

$$w[n] = \frac{1}{2}\left(1 - \cos\frac{2\pi n}{N-1}\right).$$

Windowed signal: $y_t[n] = w[n]x_t[n]$.

Spectral leakage arises because a finite segment can introduce discontinuities at its boundaries. A tapered window reduces leakage but broadens spectral peaks. It cannot create unlimited time/frequency resolution.

The shared power helper and spectral extractor use this symmetric Hann window. It should not be assumed identical to every library's default periodic Hann setting.

7.3 DFT, FFT, magnitude, and power

$$X_t[k] = \sum_{n=0}^{N-1} y_t[n]e^{-j2\pi kn/N}.$$

The DFT is the mathematical transform; the FFT is an efficient algorithm for computing it. Complex values encode both magnitude and phase.

$$A_{t,k} = |X_t[k]|, \qquad P_{t,k} = A_{t,k}^2.$$

Frequency of bin k:

$$f_k = kf_s/N.$$

With these defaults, spacing is approximately 10.77 Hz and `rfft` returns 1,025 bins. Bin spacing is not the same as guaranteed resolving power; the window and signal duration matter.

The power calculation is an unnormalized squared FFT magnitude, not a calibrated power spectral density. No physical watt or pascal unit is inferred from it.

The FMP STFT notebook provides a complementary visual explanation of short-time frequency analysis.

8. Pooling and fixed-size representations

For a frame-level quantity q_t:

$$\mu_q = \frac{1}{T}\sum_{t=1}^{T} q_t, \qquad \sigma_q = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(q_t - \mu_q)^2}.$$

`np.std` uses the population convention here, with denominator T, not $T-1$.

Pooling compresses a sequence into track-level summaries. A large standard deviation means the descriptor varies over time; it does not automatically mean noise or uncertainty.

Pooling loses order. Two recordings with very different arrangements can share means/stds. This representation cannot reliably align melodies or identify matching passages.

Default dimensions:

Group	Measurements	Stored values
MFCC	20 coefficients × mean/std	40
Spectral	4 descriptors × mean/std	8
Energy	RMS × mean/std	2
Rhythm	Tempo + onset mean/std	3
Total	Enabled groups	53
Optional tonal	12 chroma + 6 tonal coordinates, mean/std	+36

The mean-only ablation removes columns ending in `_std`. Tempo remains because it is a single estimate, not a standard-deviation column. Thus the mean-only representation has 27 values, not 26.5 or simply half of 53.

9. RMS energy

`extract_energy_features()` uses the unwindowed audio frames:

$$r_t = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} x_t[n]^2}.$$

Returned fields: `energy_rms_mean` and `energy_rms_std`.

Squaring makes negative samples contribute positively. A constant unit-amplitude signal has RMS 1; silence has RMS 0. A long sine of peak amplitude a has RMS approximately $a/\sqrt{2}$, subject to frame alignment.

RMS is a signal-strength descriptor. It is affected by recording gain, compression, padding, and channel mixing. It is not a psychoacoustic loudness score. This distinction matters when comparing professionally mastered music.

The UI's energy variation is precisely the stored RMS standard deviation. The app does not estimate a standardized loudness range.

---

10. Spectral descriptors

These descriptors use magnitude $A_{t,k}$, not power $P_{t,k}$.

10.1 Spectral centroid

$$c_t = \frac{\sum_k f_k A_{t,k}}{\sum_k A_{t,k}}.$$

It is a magnitude-weighted frequency center. Higher centroid can correspond to brighter content, but also to noise, distortion, percussion, or recording EQ. It is not fundamental pitch.

A pure 1,000 Hz tone should have centroid near 1,000 Hz, not exactly so under finite windows and floating-point effects.

10.2 Spectral bandwidth

$$b_t = \sqrt{\frac{\sum_k A_{t,k}(f_k - c_t)^2}{\sum_k A_{t,k}}}.$$

It is the weighted spread around the centroid. A concentrated tone is narrow; a mixture distributed across frequencies is broader. This is not a network bandwidth or an instrument identification result.

10.3 Spectral rolloff

For $\rho = 0.85$, find the smallest k^* satisfying

$$\sum_{k=0}^{k^*} A_{t,k} \geq \rho \sum_{k=0}^{F-1} A_{t,k}.$$

Then $o_t = f_{k^*}$.

The boundary is computed per frame and pooled across the excerpt. It is not a maximum present frequency. In this code the threshold covers **spectral magnitude**, so “85% of physical energy” would be an inaccurate description.

10.4 Zero-crossing rate

$$z_t = \frac{1}{N-1} \sum_{n=1}^{N-1} \mathbf{1}\left[\text{signbit}(x_t[n]) \boxminus \text{signbit}(x_t[n-1])\right].$$

Exact zero is treated as non-negative. There is no small-amplitude noise threshold.

ZCR measures sign transitions in the waveform, not spectral bins. It can increase for noisy or high-frequency signals but does not uniquely identify percussion. For a well-sampled sine of frequency f, a rough rate is $2f/f_s$.

Silent frames receive zero centroid, bandwidth, and rolloff through safe zero-denominator handling.

---

11. Mel filters and MFCCs

11.1 Mel scale

The custom implementation uses the HTK conversion:

$$m(f) = 2595 \log_{10}(1 + f/700), \quad f(m) = 700(10^{m/2595} - 1).$$

Mel spacing allocates relatively more detail to lower frequencies. This is an engineering representation inspired by frequency perception, not an exact model of a person's hearing.

11.2 Filter construction

Create $M + 2$ equally spaced Mel points between 0 and $m(f_s/2)$, invert to Hz, then discretize:

$$b_j = \lfloor (N+1) f_j / f_s \rfloor.$$

The code clips these to the available real FFT bins. For a triangle with left/center/right indices l, c, r:

$$W[k] = \begin{cases} (k-l)/(c-l), & l \le k < c, \\ (r-k)/(r-c), & c \le k < r, \\ 0, & \text{otherwise.} \end{cases}$$

Degenerate halves with equal boundary indices are skipped. Filters peak at one where their bins permit; there is no Slaney area normalization.

11.3 Mel power and log compression

$$E_{t,m} = \sum_k P_{t,k} W_{m,k}.$$

Raw decibel-like log power:

$$B_{t,m} = 10 \log_{10}(\max(E_{t,m}, \epsilon)).$$

Here ϵ is the smallest normal positive float64 value used by the code. Clip the complete matrix:

$$\widetilde{B}_{t,m} = \max(B_{t,m}, B_{\max} - 80).$$

For MFCC extraction, the code does **not** subtract $B_{\max}$. It clips the dynamic range while retaining absolute log-power offsets. The spectrogram display does subtract the peak; do not conflate those two operations.

11.4 DCT and cepstral coefficients

An orthonormal type-II DCT across Mel bands is

$$a_{t,c} = \alpha_c \sum_{m=0}^{M-1} \widetilde{B}_{t,m} \cos\left[\frac{\pi}{M}(m + \tfrac{1}{2})c\right],$$

with $\alpha_0 = 1/\sqrt{M}$, and $\alpha_c = \sqrt{2/M}$ for $c > 0$.

Retain $c = 0, \ldots, 19$, then pool each coefficient into mean/std. The naming starts at `mfcc_01`, which contains mathematical coefficient zero. It is energy-related and can retain recording-level effects.

Cepstral describes a transformed representation of a log spectrum; coefficients are not frequencies or notes. A coefficient may be negative.

MFCCs describe broad spectral shape rather than preserving all harmonics, phase, or note order. The project does not apply pre-emphasis, compute delta/delta-delta MFCCs, or explicitly subtract cepstral means within a track. Column standardization later is a different operation.

11.5 Custom versus Librosa MFCCs

The project's custom MFCC recipe uses HTK filters, no filter-area normalization, symmetric Hann windows, complete frames without centered STFT padding, and the above log-range rules. Librosa's default MFCC recipe has different defaults.

Changing libraries while keeping a column name does not preserve semantics. Rebuild features, scaler, PCA, vectors, and recorded results if the extraction definition changes. This is why adding Librosa display measurements did not replace stored MFCCs.

12. Stored rhythm features

12.1 Energy-rise onset envelope

From RMS r_t:

$$u_0 = 0, \qquad u_t = \max(r_t - r_{t-1}, 0).$$

Returned onset fields are the mean and standard deviation of u_t. An onset means a positive energy rise in this implementation, not a confirmed note or beat onset.

12.2 Autocorrelation and lag selection

Smooth u_t with a Gaussian of standard deviation one frame to form $\tilde{u}_t$. Compute non-negative-lag autocorrelation:

$$R[\ell] = \sum_t \tilde{u}_t \tilde{u}_{t-\ell},$$

over overlapping valid indices.

Let $f_{\text{frame}} = f_s/H$. Restrict lags to

$$\ell_{\min} = \lceil 60 f_{\text{frame}}/200 \rceil, \quad \ell_{\max} = \lfloor 60 f_{\text{frame}}/40 \rfloor,$$

also bounded by the envelope length. Choose the maximum within that range if it exceeds $10^{-6} R[0]$.

$$\widehat{\text{BPM}} = 60 f_{\text{frame}}/\ell^*.$$

There is no trained tempo model, explicit beat-time output, normalization for the shrinking lag overlap, or comprehensive confidence calibration.

12.3 Half-time/double-time ambiguity

A recording can emphasize alternating beats or subdivisions. Both a 0.5-second and 1-second recurrence may be plausible. An estimate of 60 BPM versus 120 BPM can reflect metrical ambiguity rather than completely unrelated audio.

The tests cover known pulse rates and alignment sensitivity, but real music includes syncopation, changing tempo, weak percussion, and mixed meters. Synthetic success is not proof of broad tempo accuracy.

13. Optional tonal features

Disabled by default; included here because they exist in the code.

13.1 Pitch classes

For FFT frequencies at least 27.5 Hz:

$$p(f) = 69 + 12\log_2(f/440), \qquad q(f) = \operatorname{round}(p(f)) \bmod 12.$$

A4 at 440 Hz maps to MIDI-like pitch 69 and pitch class 9. Chroma class 0 is C.

Aggregate power into each pitch class:

$$C_{t,q} = \sum_{k:q(f_k)=q} P_{t,k}.$$

Normalize by the total chroma power per frame. Empty/silent rows stay zero.

Chroma ignores octave. It can support harmony-related comparisons, but FFT-bin rounding is coarse, especially at low frequencies. Overtones, tuning deviations, and percussive content can influence it.

13.2 Tonal projection

For normalized chroma $\bar{\mathbf{C}}_t$, build six projection rows:

$$[\sin(7\pi q/6), \cos(7\pi q/6), \sin(3\pi q/2), \cos(3\pi q/2), \tfrac{1}{2}\sin(2\pi q/3), \tfrac{1}{2}\cos(2\pi q/3)].$$

Multiply chroma by the transpose of this projection. The circle pairs encode fifth/third relationships. Pool the six resulting coordinates and twelve chroma dimensions into means/stds.

This is a simple FFT-based tonal-centroid representation, not a harmonic-separated or tuning-corrected Librosa tonnetz pipeline. With tonal enabled, the default raw count would become $53 + 36 = 89$.

14. Librosa display analysis

Librosa is genuinely used for three display topics. Installed project requirements pin version 1.0.0. The linked public API pages below describe the corresponding interfaces; this report also checks the local implementation.

14.1 Spectral-flux tempo

The code forms frequency-first power $P_{k,t}$, converts it with `librosa.power_to_db(ref=np.max)`, then passes that log-power matrix to `librosa.onset.onset_strength(center=False)`.

For its default one-frame lag and no frequency-neighborhood enlargement, a useful schematic is

$$s_t \approx \frac{1}{F} \sum_k \max(L_{k,t} - L_{k,t-1}, 0).$$

It measures frequency-specific increases, not just changes in total RMS. The input here is a log-power **FFT** spectrogram; the code supplies `S`, so Librosa does not construct its default Mel spectrogram.

`librosa.feature.tempo` aggregates periodicity evidence with its default tempo prior, favoring values near 120 BPM. The code sets maximum tempo 200 but does not impose the stored estimator's explicit 40 BPM lower bound. See the tempo API.

This estimate is shown as `tempo_bpm` in the UI. `rhythm_tempo_bpm` in saved version 2 features still uses RMS autocorrelation. The two estimators must be compared as different methods.

14.2 Spectral flatness

The code passes magnitude A and leaves Librosa's power exponent at two. For floored frame power p_k, the calculation is

$$\mathrm{SF} = \frac{\exp(\frac{1}{F}\sum_k \log p_k)}{\frac{1}{F}\sum_k p_k}.$$

It compares geometric and arithmetic means. A concentrated spectrum is usually low; a flatter spectrum is higher. Sampled white noise does not need to produce exactly one, and silence is explicitly assigned zero by the wrapper. The number is not a probability of the recording being “noise.” See the flatness API.

14.3 Spectral contrast

Librosa divides the supplied magnitude spectrum into octave-related bands starting around 200 Hz and compares top/bottom quantiles, by default 2%. The wrapper limits the band count to respect the sample rate and averages contrast over bands and frames.

Away from numerical floors and dB clipping, the installed logarithmic implementation can be expressed as

$$\mathrm{contrast}_{b,t} \approx 10 \log_{10} \frac{\overline{A}_{\mathrm{top},b,t}}{\overline{A}_{\mathrm{bottom},b,t}}.$$

The factor is 10 here because the implementation applies `power_to_db` to the peak/valley summaries even though the supplied spectrum is magnitude. Do not silently replace this by a generic 20-log amplitude formula. The contrast API describes the band/quantile behavior.

High contrast suggests stronger separation between spectral peaks and valleys; it is not a recording-quality grade.

14.4 Why preserve both custom and library calculations?

The original high-level Librosa attempts were unusually slow in fresh local processes. The exact root cause was not fully isolated. Later checks with normal cache access completed quickly; the three-measurement sample benchmark took about one second. These are local observations, not universal performance claims.

The custom retrieval pipeline is already fitted and versioned. Adding supplemental library measurements gives a useful comparison without making old models incompatible.

---

15. Search-space construction

15.1 Feature selection

Selected groups are identified by prefixes. Mean-only mode removes `_std` columns. The numeric table excludes `track_id`, which remains identity metadata.

Selection precedes fitting. A disabled group contributes no scaler/PCA inputs.

15.2 Column standardization

For catalog feature j:

$$\mu_j = \frac{1}{N_c}\sum_i x_{ij}, \quad \sigma_j = \sqrt{\frac{1}{N_c}\sum_i (x_{ij}-\mu_j)^2}, \quad z_{ij} = (x_{ij}-\mu_j)/\sigma_j.$$

`StandardScaler` handles constant columns with a safe scale, yielding centered zeros. It does not guarantee robustness to outliers or semantic comparability by itself. Reference: StandardScaler.

The saved means/scales are fitted once. A query with one feature value cannot meaningfully fit its own distribution and remain comparable to the catalog.

15.3 Per-group L2 normalization and weights

For standardized group vector $\mathbf{z}_{i,g}$:

$$\mathbf{h}_{i,g} = \begin{cases} \mathbf{z}_{i,g}/\|\mathbf{z}_{i,g}\|_2, & \|\mathbf{z}_{i,g}\|_2 > 0, \\ \mathbf{0}, & \text{otherwise.} \end{cases} \qquad \mathbf{w}_{i,g} = \alpha_g \mathbf{h}_{i,g}.$$

Concatenate group blocks.

This balances groups with unequal dimensions, but removes each group's standardized magnitude. Magnitude might contain useful information; equalizing groups is a modeling decision, not a universal fairness law.

If every group is nonzero and PCA is skipped:

$$\cos(\mathbf{w}_i, \mathbf{w}_j) = \frac{\sum_g \alpha_g^2 \, \mathbf{h}_{i,g}^T \mathbf{h}_{j,g}}{\sum_g \alpha_g^2}.$$

Consequently a weight of two contributes four times the dot-product weight of one under these conditions. Zero groups change the denominator; PCA changes the geometry further.

15.4 PCA

Let W be the weighted catalog matrix and $\bar{\mathbf{w}}$ its column mean. Center it:

$$W_c = W - \mathbf{1}\bar{\mathbf{w}}^T.$$

A singular value decomposition is

$$W_c = USV^T.$$

Keep the first P directions V_P:

$$Y = W_c V_P.$$

PCA is based on variance, not labels. It may preserve irrelevant recording variation and discard useful low-variance distinctions. Component columns are mixtures of original columns; `pca_01` is not an MFCC coefficient.

The current code uses the full SVD solver. Explained variance for component p is approximately

$$\lambda_p = s_p^2/(N_c - 1).$$

Reference for API conventions: scikit-learn PCA.

15.5 Whitening and final normalization

Whitening rescales centered projected coordinates:

$$Y_{i,p}^{\text{white}} = Y_{i,p}/\sqrt{\lambda_p}.$$

The implementation rejects extremely small retained variance before whitening. Whitening can amplify weak/noisy directions; retaining high total variance does not imply high retrieval relevance.

Finally:

$$\mathbf{v}_i = \mathbf{y}_i / \|\mathbf{y}_i\|_2$$

for nonzero rows. Zero rows stay zero but are not valid queries.

The saved current model retains about **99.07%** of weighted-feature variance in 40 components. This is an unsupervised variance ratio, not accuracy or information completeness.

15.6 Query consistency

The query feature dictionary is reordered with `models["feature_columns"]`, then uses saved `transform` methods. This is crucial because matrix operations depend on column position.

Always pair the vector file with models from the same fit. The code detects differing output column schemas; it does not cryptographically prove two same-shaped files belong to the same fit.

---

16. Cosine retrieval and genre voting

16.1 Similarity

$$s(\mathbf{q}, \mathbf{v}_i) = \frac{\mathbf{q}^T \mathbf{v}_i}{\|\mathbf{q}\|_2 \|\mathbf{v}_i\|_2}.$$

For normalized vectors, it equals a dot product. The implementation uses `cosine_similarity`, so it also supports non-unit vectors. Reference: cosine similarity API.

Rank descending. A score near one means aligned vectors; zero means orthogonality; negative means opposite directions in this representation. PCA centering means negative values can occur naturally.

For unit vectors:

$$\|\mathbf{q} - \mathbf{v}_i\|_2^2 = 2 - 2s(\mathbf{q}, \mathbf{v}_i).$$

Thus unit-vector Euclidean distance and cosine similarity yield equivalent orderings, apart from ties/numerical effects.

16.2 Self-exclusion and duplicates

Dataset-ID search excludes the query ID. File search can exclude an exact existing catalog path. External uploads receive a new temporary ID; uploaded copies of a catalog recording are not automatically recognized as duplicates.

The Streamlit genre estimator can therefore include an identical catalog recording if a user uploads it. That is not an independent evaluation and should not be treated as one.

16.3 Genre vote

For the up-to-ten closest valid neighbors $\mathrm{N}(q)$, count each label:

$$V_g = \sum_{i \in \mathrm{N}(q)} \mathbf{1}[y_i = g].$$

Choose maximum V_g. A count tie is broken by summed similarity:

$$S_g = \sum_{i \in \mathrm{N}(q),\, y_i = g} s(q, i).$$

This is a majority-first rule, not a similarity-weighted sum as the primary vote. Remaining exact ties follow grouped label order.

The UI's neighbor count is fixed in `analyze_audio` at ten rather than exposed as a slider. It is not currently taken from the CLI's `retrieval.k` setting.

All queries receive a catalog label if valid labeled matches exist. No unknown-genre rejection threshold or calibrated confidence model is implemented. A jazz/rock hybrid or an out-of-catalog genre may still be assigned one of the ten catalog labels.

---

17. Mel spectrogram and displayed measurements

The display uses the same custom Mel-power calculation as the MFCC pipeline. It converts to relative dB:

$$D_{t,m} = 10 \log_{10}(\max(E_{t,m}, \epsilon)) - 10 \log_{10}(E_{\max}),$$

clipped to $[-80, 0]$. Completely silent input produces a matrix filled with -80.

Horizontal axis: time inside the prepared excerpt.
Vertical axis: Mel-spaced coordinates, labeled in Hz.
Color: relative log power. Brighter is stronger within that excerpt, not universally louder across different uploads.

Frame-center time:

$$\tau_t = (tH + N/2)/f_s.$$

The rendering places pixel edges half a hop/band from their centers. Frequency tick labels are converted onto the HTK Mel coordinate system. Very low/high boundaries are not additional measured filter bands.

A PNG is rendered in memory and kept in session state. The download button returns those same bytes.

Display	Origin	Important distinction
Predicted genre	Catalog neighbor vote	No calibrated classifier confidence
File duration	Original header	Not the excerpt duration
Original sample rate	Original header	Not the analysis sample rate
Channels	Original header	Features are mono
Estimated tempo	Librosa spectral flux	Different from stored RMS tempo
Spectral brightness	Centroid mean	Not pitch
RMS energy	Frame RMS mean	Not LUFS
Frequency spread	Bandwidth mean	Not hardware/network bandwidth
High-frequency rolloff	Magnitude rolloff mean	Not max frequency
Zero-crossing rate	Sign transitions	Sensitive to frequency/noise
Sample peak	Prepared waveform peak	Not true peak
Energy variation	Frame RMS std	Not standardized loudness range
Onset strength	Stored RMS-rise mean	Not Librosa beat confidence
Spectral flatness	Librosa	Not probability of noise
Spectral contrast	Librosa	Not recording quality

There are 14 technical measurements plus genre. The Mel image is an additional visual output. Its presence does not imply these display-only fields are all used in retrieval.

---

18. Evaluation definitions and worked example

18.1 Binary relevance

For query genre y_q and result label y_r:

$$\mathrm{rel}(q, r) = \mathbf{1}[y_q = y_r].$$

Genre agreement is the declared proxy, not a listening-based truth about similarity.

Let $r_j \in \{0, 1\}$ denote relevance at rank j, and R_q the number of relevant catalog candidates excluding the query.

18.2 Precision, recall, AP, and nDCG

$$P@K = \frac{1}{K} \sum_{j=1}^{K} r_j.$$

$$\mathrm{Recall}@K = \begin{cases} \sum_{j=1}^{K} r_j / R_q, & R_q > 0, \\ 0, & R_q = 0. \end{cases}$$

For this project's truncated AP convention:

$$AP@K = \frac{\sum_{j=1}^{K} P@j \, r_j}{\min(K, R_q)}$$

when the denominator is nonzero; otherwise return zero. This convention matters because other implementations divide by all R_q.

MAP@K is the mean AP@K across queries.

$$DCG@K = \sum_{j=1}^{K} \frac{r_j}{\log_2(j + 1)}.$$

With $I = \min(K, R_q)$:

$$IDCG@K = \sum_{j=1}^{I} \frac{1}{\log_2(j + 1)}, \quad nDCG@K = DCG@K / IDCG@K.$$

If ideal DCG is zero, the code returns zero.

18.3 Worked ranking example

Suppose $K = 5$, relevance is $[1, 0, 1, 1, 0]$, and there are four relevant candidates in the catalog.

Precision: $3/5 = 0.6$.

Recall: $3/4 = 0.75$.

Relevant ranks are 1, 3, and 4, so AP is $(1 + 2/3 + 3/4)/4 \approx 0.6042$.

DCG is $1 + 1/\log_2(4) + 1/\log_2(5) \approx 1.9307$.

Ideal DCG has four relevant positions and is approximately 2.5616.

nDCG is approximately 0.7537.

Moving a relevant result upward can improve AP/nDCG without changing Precision@5.

18.4 Random baseline

For genre count n_g among N_c evaluated tracks:

$$\mathrm{E}[P@K \mid y_q = g] = \frac{n_g - 1}{N_c - 1}.$$

Randomly selecting K different candidates has this expected fraction for any valid K. The code calculates expectation directly rather than performing random trials.

Global expected precision weights queries equally:

$$\bar{P}_{\text{random}} = \frac{1}{N_c} \sum_g n_g \frac{n_g - 1}{N_c - 1}.$$

Lift is $\bar{P}_{\text{system}} / \bar{P}_{\text{random}}$ if the denominator is positive. Lift is a ratio on this metric, not “times better at understanding music.”

The evaluator requires nonzero vectors and enough candidates for exactly K results. This keeps denominators consistent.

---

19. Recorded results and interpretation

19.1 Current combined PCA baseline

Evidence: `data/evaluation/current/summary.json`.

Metric	Recorded value
Tracks	999
K	10
Mean Precision@10	0.486186
Mean Recall@10	0.049157
MAP@10	0.389946
Mean nDCG@10	0.521450
Expected random Precision@10	0.099099

Metric	Recorded value
Lift over random	4.906061

Interpretation: about 4.86 of ten results share the query genre on average. Recall is low partly because each query has about 99 relevant genre-mates but only ten results are requested. The maximum recall at K=10 is only about 10/99 for most genres.

These numbers do not measure the deployed genre estimator's external-upload accuracy, tempo accuracy, calibrated confidence, or subjective musical relevance.

19.2 Genre breakdown

Evidence: `data/evaluation/current/by_genre.csv`.

Genre	Tracks	Mean Precision@10
Classical	100	0.7690
Metal	100	0.6360
Pop	100	0.6160
Blues	100	0.5770
Jazz	99	0.4657
Hiphop	100	0.4210
Reggae	100	0.3870
Country	100	0.3650
Disco	100	0.3240
Rock	100	0.3010

A plausible interpretation is that some genres have more distinctive timbral/production distributions. This remains a hypothesis; the scores alone do not establish that reason. Dataset artifacts could also contribute.

19.3 Eight ablations

Evidence: `config/ablation.yaml` and `data/evaluation/ablations/comparison.csv`.

ID	Representation	Dimensions	PCA	Precision@10
A	MFCC only	40	None	0.502703
B	Spectral only	8	None	0.317618

ID	Representation	Dimensions	PCA	Precision@10
C	Rhythm + energy	5	None	0.258959
D	MFCC + spectral	48	None	0.483483
E	All enabled groups, equal weights	53	None	0.462763
F	All groups, chosen unequal weights	53	None	0.463063
G	All groups, equal weights	40	Whitened	0.486186
H	All groups, no std columns	27	None	0.426126

The F weights are MFCC 1.0, spectral 0.8, rhythm 1.2, and energy 0.6. Its name says “tuned,” but no automated search or independent validation-based tuning was implemented. Treat them as manually chosen trial weights.

A beats G by approximately 0.0165 absolute precision (1.65 percentage points). This establishes a descriptive difference in this catalog, not statistical significance or future performance.

G improves over E in this snapshot. It changes **both** PCA reduction and whitening; the experiment does not separately isolate their effects. Add PCA-without-whitening to disentangle them.

H removes all standard-deviation columns together. The result suggests those combined variability descriptors help the current setup, but does not identify which particular std columns matter.

19.4 What to claim responsibly

A defensible statement is:

> We implemented an interpretable content-based music retrieval pipeline and compared eight feature-space variants using same-genre catalog relevance. The MFCC-only variant achieved the highest observed Precision@10 among these configurations.

An unsupported statement is:

> The system recognizes any song's genre with 50% accuracy.

Retrieval precision and majority-vote classification accuracy are different measurements.

20. Implementation, configuration, and deployment

20.1 Configuration

Dataclasses group audio, feature, pooling, space, retrieval, and path settings. YAML is read with `safe_load`. Nested overrides merge into copied defaults, so one experiment does not mutate another's base.

Configuration controls extraction settings, selected groups, pooling columns, weights, PCA, and catalog paths. The UI loads the saved model's extraction settings for the actual query, avoiding mismatches with newly edited YAML.

Not every declared option is arbitrary: feature preparation currently requires mono, and optional tonal extraction supports 12 chroma classes.

20.2 UI state and caching

Streamlit reruns the script when widgets change.

`cache_data` avoids repeatedly loading immutable tables.

`cache_resource` shares trusted fitted model objects.

File modification timestamps are cache arguments so updated artifacts invalidate caches.

Session state stores the completed analysis dictionary and PNG.

An audio-content hash plus an analysis-schema prefix clears stale results.

A button triggers analysis rather than every UI rerun.

Uploads use a temporary file removed after analysis.

The bundled sample is read from its existing local path.

The preview player uses the complete selected recording. The spectrogram shows only the prepared excerpt. Local caching is a performance/state mechanism, not a privacy guarantee for arbitrary hosting settings.

20.3 Commands

Run from the repository root:

```
python -m src.seekaudio ingest
python -m src.seekaudio extract
python -m src.seekaudio build-space
python -m src.seekaudio search blues.00000 --k 5
python -m src.seekaudio search-file "C:\music\query.wav" --k 5
python -m src.seekaudio evaluate --k 10
python -m src.seekaudio ablate
```

```
python -m src.seekaudio run
python -m streamlit run streamlit_app.py
python -m pytest tests -q
```

`run` is the complete rebuild/evaluation workflow. `build-space --components 0` skips PCA. After editable package installation, `python -m seekaudio` and `seekaudio` are alternative entrypoints.

20.4 Deployment

`scripts/prepare_streamlit.py` creates a small bundle containing runtime source, requirements, cloud settings, manifest, vectors, models, and an optional sample. Raw audio, feature tables, and tests are not copied.

Pinned library versions matter because saved Python objects can be incompatible with different versions. Python 3.12 is the intended cloud runtime. Dependencies include SoundFile, SciPy, NumPy, Pandas, scikit-learn, Matplotlib, Streamlit, PyArrow, YAML, Joblib, and Librosa.

The repository's raw dataset is already tracked; adding a Git ignore rule does not erase it from history. The separate deployment repository avoids copying the dataset without rewriting history.

Only publish bundled commercial audio when redistribution is permitted. The app's upload path works without the sample.

Live URL supplied during deployment: seekcode-audio.streamlit.app. This report does not independently certify the current server's commit or uptime.

---

21. Tests, complexity, and reproducibility

21.1 Testing strategy

The most recent complete development test run passed 66 tests. This count is a snapshot, not a permanent claim that future changes automatically pass.

Tests use controlled signals and small catalogs:

Temporary WAV files check metadata and save/load behavior.

Known tones check centroid, narrow bandwidth, rolloff, and ZCR.

Silence checks stable zeros and dark spectrograms.

Pulse trains check tempo estimates.

Noise versus tone checks Librosa flatness.

Synthetic vectors check cosine order, self-exclusion, and zero handling.

Saved-model tests reproduce query transforms and rankings.

Known rankings check metric denominators and rank penalties.

Pipeline integration tests create artifacts, reports, and plots.

UI smoke checks additionally exercise sample/upload analysis, state persistence, and the expected display fields. Automated calculations do not replace perceptual listening checks.

21.2 Computational costs

For T frames and window size N, FFT work is approximately $O(TN \log N)$. The Mel multiplication costs approximately $O(TFM)$ with the dense implementation.

Raw framing views are cheap, but temporary windowed matrices and FFT outputs allocate memory. `load_audio` currently decodes the complete original recording before cropping, so processing a long high-rate upload uses more memory than its final 30-second excerpt implies.

For one D-dimensional query and N_c candidates, cosine comparisons cost about $O(N_c D)$, and the full sort about $O(N_c \log N_c)$. The current search is brute force, not an approximate-nearest-neighbor index.

Evaluation repeats this search for every catalog query, giving roughly quadratic catalog-comparison work. That is practical for 999 tracks but would need redesign for a very large catalog.

Full PCA SVD cost depends on rows/columns; it is inexpensive at 999 × 53 relative to decoding/extraction. Do not extrapolate local timings to every cloud instance.

21.3 Reproducibility checklist

Record:

Source commit and extraction feature version.

Python/dependency versions.

Dataset provenance and unreadable/excluded tracks.

Full extraction recipe and feature-column order.

Group selection, weights, pooling, PCA/whitening choices.

Paired vectors/models from the same fit.

K, relevance definition, self-exclusion, AP convention.

Per-track scores, not just a single aggregate.

Whether evaluation is catalog-based or held out.

Random seeds for generated tests or future resampling experiments.

Full SVD avoids randomized-PCA seed variation, but numerical libraries/platforms can still introduce tiny floating-point differences. Compare with tolerances when appropriate.

---

22. Limitations and next research steps

22.1 Current limitations

Genre is an imperfect proxy for acoustic relevance.

Catalog-fitted transforms see every evaluated track's features.

GTZAN repetitions/artist overlap/label issues have not been fully audited.

External genre prediction has not been independently scored.

A fixed centered excerpt can miss the most representative section.

Mean/std pooling discards musical time order.

RMS-based stored rhythm is simple and metrically ambiguous.

MFCCs and energy can depend on mastering/gain.

Group normalization can discard meaningful magnitude information.

Whitening can amplify small-variance noise.

Custom HTK MFCCs are not interchangeable with library defaults.

The app has no open-set/unknown-genre decision.

Tonal features are approximate and optional.

Uploaded catalog copies are not fully duplicate-detected.

The saved-artifact format is version-sensitive; Joblib files must be trusted.

File identity, vector/model provenance hashes, and resource limits could be enforced more strongly.

22.2 A more rigorous next evaluation

For independent query generalization:

Identify duplicate recordings and, where possible, group by artist.

Split development and test queries at a meaningful grouping level.

Fit scaler/PCA only on the allowed reference/training data.

Select feature settings using development queries.

Freeze choices before evaluating independent test queries.

Define whether test songs are excluded from the searchable catalog.

Measure retrieval metrics and separately genre-vote accuracy/macro-F1.

Audit confusion matrices and per-class results.

Add paired uncertainty estimates and listening-based judgments.

Catalog evaluation can be a legitimate transductive retrieval setting if declared. It should not be presented as an inductive held-out test.

22.3 Useful ablations not yet implemented

Compare unwhitened versus whitened PCA, several component counts, robust scaling, weighted/no-weighted groups, custom versus library MFCCs under matched settings, multiple excerpt locations, and rhythm definitions. Use independent development data for selection.

Temporal sequence representations, DTW, learned audio embeddings, larger datasets, calibrated rejection, and approximate indexing are possible research extensions. They are not features of the current implementation.

---

23. Glossary and misconceptions

Term	Practical meaning
Sample	One numeric amplitude observation
Sample rate	Samples per second
Bit depth	Quantization precision of encoded PCM; not the same as NumPy dtype
Channel	One audio signal stream
Waveform	Samples arranged over time
Frame	Short block of samples
Hop	Distance between successive frame starts
Window	Weights applied to a frame before spectral analysis
FFT bin	One sampled transform frequency position
Nyquist	Half the sample rate
Aliasing	High frequencies folding into representable frequencies
Leakage	Spectral energy spreading because a finite window truncates a signal
Magnitude	Absolute value of a complex FFT coefficient
Power	Squared magnitude in this implementation
Spectrogram	Time-frequency matrix rendered as an image
Mel	Nonlinear frequency-coordinate scale

Term	Practical meaning
MFCC	DCT coefficients of log-Mel power
Timbre	Sound character beyond pitch and overall level
Chroma	Pitch class independent of octave
Onset	A detected increase/attack; definition depends on the algorithm
Beat	Perceived periodic musical pulse
Tempo	Beat-rate estimate in BPM
Pooling	Compressing a sequence into summary values
Feature	A measurable numeric descriptor
Vector	Ordered collection of feature/component values
Standardization	Centering/scaling each feature across catalog tracks
L2 normalization	Dividing a vector by Euclidean length
PCA	Unsupervised directions maximizing variance
Whitening	Rescaling component variances
Cosine similarity	Normalized vector dot product
Neighbor	A catalog item ranked close under the chosen representation
Relevance	Declared criterion for evaluating a result
Calibration	Agreement between claimed probabilities and observed frequencies
Ablation	Controlled removal/change of a component to study contribution
Leakage in ML	Evaluation information influencing fitting/selection beyond the declared protocol
Inference	Applying fitted transformations/decision rules to new input
Artifact	Saved data/model/report produced by a pipeline
Serialization	Saving objects for later loading
Brute force	Comparing against every candidate
Cache	Reusing a previously loaded/computed object

Common mistakes to avoid:

“0.49 precision” is not “49% correct genre predictions.”

A 0.8 cosine score is not an 80% probability of musical similarity.

99% PCA variance retention is not 99% accuracy.

A bright spectrogram is not necessarily louder than another separately normalized image.

Centroid is not the note pitch.

ZCR is not a genre detector by itself.

A 123 BPM display and 63 BPM stored estimate need not indicate a simple coding bug.

More features and more complicated algorithms do not guarantee better retrieval.

Using Librosa does not make the pipeline a neural network.

A successful song demo is evidence of a working path, not a controlled benchmark.

---

24. References and evidence

Repository evidence

Configuration and ablation definitions.

Current aggregate metrics.

Current genre metrics.

Ablation comparison.

Detailed method atlas.

Annotated offline coordinator.

Annotated live coordinator.

Primary technical resources

FMP notebook collection: MIR foundations and worked algorithms.

FMP short-time Fourier transform: time-frequency analysis.

Sturm's GTZAN analysis resources: dataset cautions and research references.

SciPy resample_poly: rational resampling.

scikit-learn StandardScaler.

scikit-learn PCA.

scikit-learn cosine similarity.

Librosa tempo, flatness, and contrast: public API references. The repository pins 1.0.0; inspected installed behavior takes precedence if versions differ.

The equations in implementation sections are reconstructions of this repository's calculations. References support background/API interpretation rather than supplying undocumented claims about the project's results.

PART II — LEARNING GUIDE

seekaudio: from-scratch learning and reconstruction guide

This guide is for you as the developer, not only for a portfolio visitor. Its goal is to help you understand, reproduce, explain, and extend the system rather than memorize names.

Read it alongside the technical report, method reference, and annotated Python files. The report explains the current implementation; this guide orders the prerequisites and practice.

---

1. How to use this guide

Use three passes:

Run and observe: follow a real recording through existing methods; inspect shapes and units.

Rebuild small pieces: implement one calculation with a controlled signal, compare it with the project, and write down why it works.

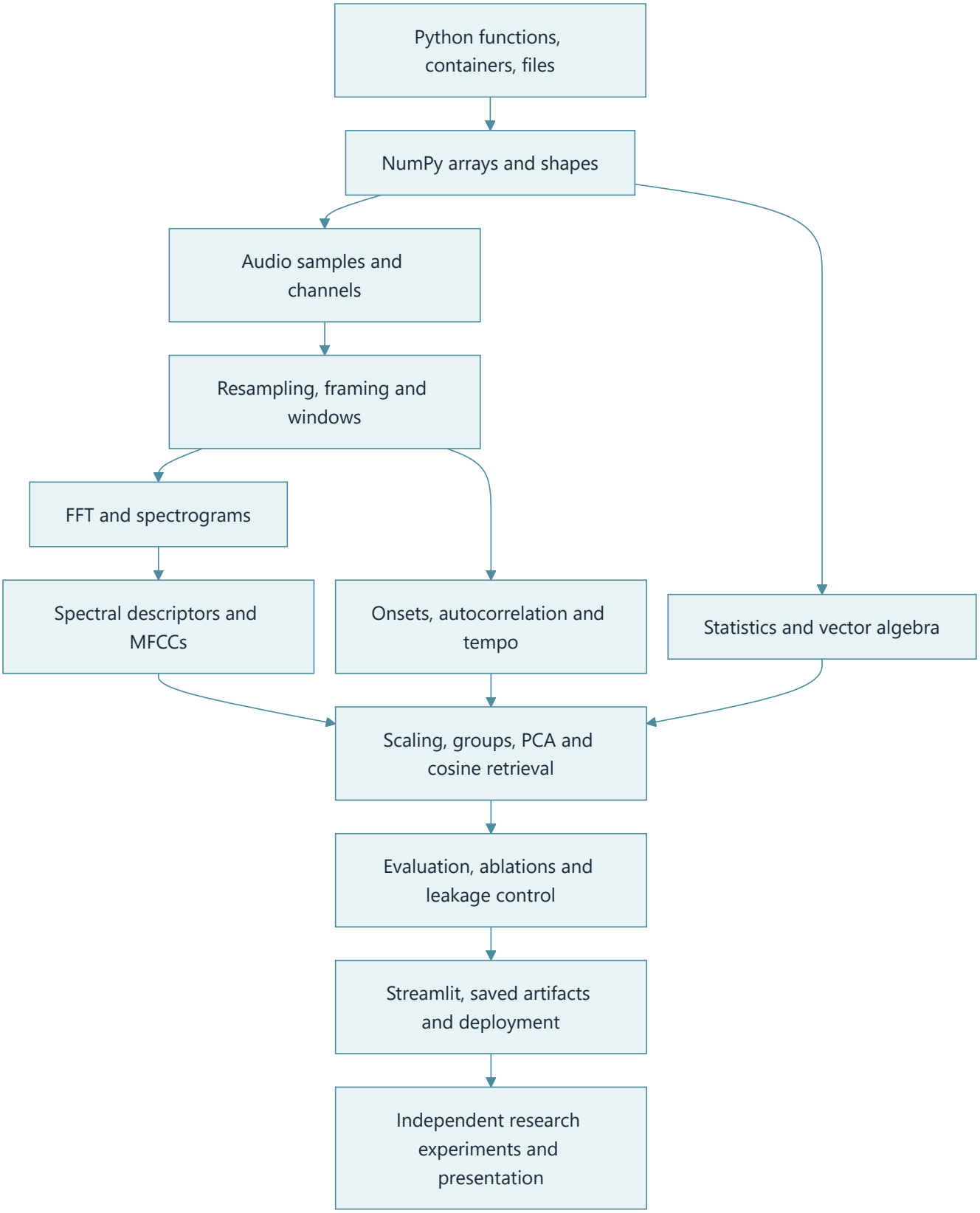
Investigate choices: run a controlled experiment and explain the evidence, including where you are uncertain.

Keep a learning notebook with four headings for every concept: **definition, equation, code, failure cases**. If you cannot explain the unit or array shape of a value, pause before adding more code.

The suggested sequence has twelve stages. A stage can take a few evenings or longer depending on your background. Treat the checkpoints as the completion criterion rather than an arbitrary calendar deadline.

Do not start with a neural model simply because the project concerns AI. First understand how the existing representation creates its similarity decisions.

2. Prerequisite map



There are two prerequisite branches: programming/mathematics and audio/signal processing. You need both before you can interpret a search score.

3. Establish a reproducible workspace

Work from the repository root. Use Python 3.12, matching the deployment environment.

```
python --version
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pip install -e ".[dev]"
python -m pytest tests -q
python -m streamlit run streamlit_app.py
```

If PowerShell blocks activation, you can use the environment interpreter directly:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pytest tests -q
```

Avoid changing system-wide execution policies just to run the project.

Pinned versions protect saved-model compatibility. Use the sample's existing local file rather than adding download logic to every experiment. Do not commit credentials, large scratch outputs, or private recordings.

The `test` branch is annotated for study. Before changing behavior, keep a clear baseline commit. Saving a new feature recipe requires new artifacts, not merely a new column name.

4. Stage 1: refresh practical Python

Learn

Variables, expressions, `if`, `for`, functions, return values, lists, sets, dictionaries, indexing, imports, exceptions, `Path`, and basic type hints.

Understand these project idioms:

`str | Path` permits either type of path argument.

`dict[str, float]` describes keys/values; it does not validate values at runtime.

`-> pd.DataFrame` describes a return type.

`**settings` unpacks a dictionary into keyword arguments.

`features.update(other)` merges fields into a dictionary.

`with` scopes resources such as an open file or temporary folder.

`if __name__ == "__main__"` prevents importing a module from running its command.

`@dataclass` organizes related configuration values.

A leading underscore suggests an internal helper, not enforced access control.

Read

`ingest.py`, `config.py`, and `__main__.py` before the heavier numerical files.

Practice

Write a small function accepting a folder and returning only WAV paths. Then make it reject a missing folder with a useful exception. Construct one dictionary containing a track ID and technical fields. Do not load all music yet.

Experiment with `Path.stem`, `Path.parent.name`, and `Path.suffix.lower()` on the same file.

Checkpoint

Explain why discovering a `.wav` path does not guarantee a readable WAV file. Show the difference between defining a function and calling it. Explain exactly what `**metadata` adds to a manifest row.

Primary resource: Python 3.12 tutorial.

---

5. Stage 2: NumPy, Pandas, and shapes

Learn

NumPy arrays, dtypes, slicing, `shape`, `ndim`, transpose, axes, broadcasting, vectorized calculations, and copying versus views. Learn Pandas rows/columns, filtering, `groupby`, ID-based joins, and Parquet read/write.

Do not skip `axis`:

Frames are `(frames, samples_per_frame)`.

Averaging over `axis=1` produces one value per frame.

MFCC output is `(coefficients, frames)`.

Librosa spectra are `(frequency_bins, frames)`.

`track_id` must not become a numeric model input.

Practice

```
import numpy as np
```

```
frames = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
print(frames.shape)
print(frames.mean(axis=0))
print(frames.mean(axis=1))
print(frames.T.shape)
```

Predict every output before running it. Next, create a three-track DataFrame, reorder its rows, and join metadata by ID. Demonstrate that joining by row position would be wrong.

Checkpoint

Explain why a waveform is not a feature table. Given `(1288, 1025)`, say which axis means time and which means frequency. Explain why `features.columns` order must be saved.

Useful reference: NumPy absolute beginners guide.

---

6. Stage 3: mathematics you actually need

Learn

Arithmetic means, variance/std, square roots, logarithms, vectors, dot products, L2 norms, matrices, transpose, covariance, eigenvectors, and SVD at an intuitive level.

Learn notation rather than treating it as decoration:

$\sum$: add values.

$\sqrt{\cdot}$: square root.

$\|\mathbf{x}\|_2$: Euclidean length.

$\mathbf{x}^T\mathbf{y}$: dot product.

$\mathbf{1}[\text{condition}]$: one if true, otherwise zero.

$\arg\max$: index/value location of the largest score.

$\log_{10}$ versus $\log_2$: different bases used for different purposes.

Practice

For $[3, 4]$, calculate norm 5 and normalized vector $[0.6, 0.8]$. Calculate the cosine between $[1, 0]$, $[0, 1]$, and $[-1, 0]$. Compute a mean/std from three values by hand.

Compare population std (divide by N) and sample std (divide by N-1). The project uses population pooling/std scaling and sample-variance PCA quantities.

Checkpoint

Explain why dividing by a zero norm is invalid and how keeping the zero row unchanged avoids NaNs. Explain why negative PCA/MFCC values are not inherently errors.

---

7. Stage 4: understand digital audio

Learn

Continuous versus sampled signals, PCM, sample rate, bit depth, channels, duration, Nyquist, aliasing, amplitude, dBFS, clipping, and floating-point sample representations.

Differentiate **sample rate** from **signal frequency**. A 1,000 Hz tone can be represented by many different sample rates if they are sufficient.

Practice

Generate a tone:

```
import numpy as np

sample_rate = 8_000
seconds = 1.0
frequency = 440
time = np.arange(round(sample_rate * seconds)) / sample_rate
waveform = 0.5 * np.sin(2 * np.pi * frequency * time)
print(waveform.shape)
```

Plot a tiny time section, not all samples at once. Listen to a safely scaled saved waveform if desired. Compare stereo channels with identical versus opposite signs and observe mono averaging.

Checkpoint

Explain why a 30-second prepared clip has 661,500 samples. Explain what disappears when 48 kHz audio becomes 22.05 kHz audio. Explain why a full-file duration of 5:56 and a 30-second spectrogram can both be correct.

Read `features/audio.py` and the audio-preparation chapter of the report.

8. Stage 5: framing, windows, and the FFT

Learn

Local stationarity, overlapping frames, frame/hop length, Hann window, spectral leakage, DFT/FFT, complex values, magnitude, power, bin frequency, and phase.

Calculate default window duration, hop duration, bin spacing, and frame count yourself. Confirm **1,288 complete frames**, not a rounded estimate from seconds times frame rate.

Practice

Implement a readable framing loop in a separate exercise:

```
frames = []
for start in range(0, len(waveform) - frame_length + 1, hop_length):
    frames.append(waveform[start:start + frame_length])
frames = np.array(frames)
```

Compare this with `split_into_frames`. The project helper uses a memory-sharing view for efficiency; your loop is valuable for understanding.

Calculate an FFT of a bin-aligned sine, then a non-bin-aligned sine. Compare no window versus Hann. Increase N while holding sample rate fixed and observe the change in time/frequency tradeoff.

Checkpoint

Explain why FFT bin spacing is not a guarantee of perfect resolution. Explain why `rfft` has N/2+1 bins for even N. Identify the samples omitted at the end of the prepared clip.

Primary resource: FMP STFT notebook.

9. Stage 6: rebuild energy and spectral descriptors

Learn

RMS, magnitude weighting, centroid, bandwidth, cumulative rolloff, sign transitions, silence handling, and frame pooling.

Practice in order

Implement RMS with a small loop, then compare with NumPy.

Calculate centroid for an isolated tone.

Calculate bandwidth around the centroid.

Find the 85% cumulative-magnitude crossing.

Count sign changes and divide by N-1.

Summarize frame measurements using mean/std.

Use three controlled inputs: silence, a sine wave, and deterministic noise. Then add two tones and vary their amplitudes.

Expected observations

A tone's centroid is close to its frequency, bandwidth is narrow but not necessarily zero, and rolloff is nearby. Silence should not produce undefined values. Noise can increase spread and ZCR. The exact results depend on the signal and window.

Checkpoint

Explain why ZCR is computed from raw frames rather than the FFT. Explain why centroid is not pitch. State whether this project's rolloff uses magnitude or power.

Read `energy.py`, `spectral.py`, and their tests. Run focused tests rather than rebuilding the entire catalog after each exercise.

```
python -m pytest tests/test_energy.py tests/test_spectral.py -q
```

---

10. Stage 7: Mel spectrograms and MFCCs

Learn

HTK Mel conversion, triangular filter banks, matrix multiplication, power-to-dB conversion, numerical floors, dynamic-range clipping, DCT, cepstral coefficients, and pooling.

Remember that the project computes custom MFCCs; its Librosa use is currently supplemental display analysis.

Reconstruction sequence

Plot Hz versus Mel for a range of frequencies.

Generate M+2 boundary points.

Map boundaries to FFT bins.

Build and plot a few triangles.

Multiply frame power by the filter matrix.

Apply log compression and an 80 dB floor relative to the maximum.

Calculate DCT coefficients.

Retain twenty coefficients and pool them.

Inspect the shape after every step. An axis mistake can still produce plausible-looking numbers, so shape checks are essential.

Practice questions

Why M+2 points for M filters? What happens when two low-frequency boundaries map to the same FFT bin? Why is log(0) a problem? What does DCT coefficient zero contain? Why does `mfcc_01` not mean the first nonzero-frequency component?

Compare the MFCC log matrix with the displayed spectrogram matrix. The display subtracts the maximum; the MFCC code does not.

Checkpoint

Draw the MFCC flow from memory and explain what each arrow changes. Describe why changing to Slaney-normalized filters requires rebuilding the fitted catalog.

---

11. Stage 8: rhythm, tonal concepts, and Librosa

Learn

Onset versus beat, autocorrelation, lag-to-BPM conversion, Gaussian smoothing, spectral flux, tempo priors, octave bands, spectral flatness, spectral contrast, pitch classes, and chroma.

Practice

Generate pulses every 0.5 seconds. Calculate the expected tempo of 120 BPM. Inspect the RMS-rise envelope and its autocorrelation peaks. Then emphasize alternating pulses to create half-time ambiguity.

Compare:

Stored `extract_rhythm_features`, using RMS rises.

Display `extract_librosa_characteristics`, using spectral-flux tempo.

A manually documented tempo reference, when reliable.

Record disagreements without assuming either estimate is automatically correct.

For optional chroma, generate A4 at 440 Hz and A5 at 880 Hz. Both should emphasize pitch class A. Generate mixed pitches and observe changes in chroma.

Checkpoint

Explain why the same song can produce 63 BPM and 123 BPM estimates. Explain the role of Librosa's prior and why its maximum-tempo setting is not the same as the stored estimator's lower-and-upper interval.

Explain flatness and contrast without calling either a probability or a quality rating.

---

12. Stage 9: build the search representation

Learn

Feature tables, fitting versus inference, StandardScaler, constant features, group normalization, weight effects, PCA centering, explained variance, whitening, and final L2 normalization.

Practice

Start with a hand-made three-track table, not 999 recordings. Give one column values around 2,000 and another around 0.1. Observe unscaled Euclidean influence, then standardize.

Normalize two feature groups separately and change weights. Verify that doubling a unit group's weight produces a squared factor in the no-PCA cosine contribution.

Build a toy correlated table. Compare PCA with and without whitening and with fewer components. Plot the transformed points. Observe that PCA cares about variance, not genre.

Critical exercise: saved transformation

Fit a scaler on the small catalog. Apply `transform` to a new row. Deliberately fit a second scaler on the query alone and observe the invalid comparison. Repeat this reasoning for PCA.

Checkpoint

Explain the difference between per-column standardization, per-group normalization, and final vector normalization. Explain why the order matters and why the query cannot refit either model.

Read `space.py`, `test_space.py`, and the query-reuse test in `test_pipeline.py`.

13. Stage 10: retrieval, labels, and evaluation

Learn

Cosine similarity, nearest-neighbor ranking, stable ties, self-exclusion, candidate sets, binary relevance, Precision@K, Recall@K, AP/MAP, nDCG, and random baselines.

Practice

Construct four vectors with known directions and predict their scores before running retrieval. Use different IDs from row indices. Shuffle the manifest and confirm label joins remain correct.

Calculate the report's example relevance list $[1, 0, 1, 1, 0]$ by hand. Make a second list with the same three relevant results lower down. Explain which metrics change.

Use a small imbalanced genre catalog to derive the random expectation. Do not assume random precision is always 0.1; it depends on label counts.

Classification versus retrieval exercise

For one query, write down:

Neighbor IDs and labels.

Same-genre retrieval precision.

Majority-vote predicted genre.

Whether that final genre prediction equals a separately known ground truth.

A query can have a correct majority label with poor retrieval precision, or vice versa. Averaging those outcomes creates different metrics.

Checkpoint

Explain why excluding the query itself matters. Explain why an uploaded copy of a catalog track can still leak through a demonstration. Explain why recall near 0.05 at K=10 is interpretable when there are about 99 relevant candidates.

14. Stage 11: reproduce the experiments

Learn

Ablation, controlled comparisons, hyperparameters, confounders, independent validation, paired comparisons, and cautious result interpretation.

Practice

Read `config/ablation.yaml` and predict each experiment's dimensionality. Check A=40, B=8, C=5, D=48, E/F=53, G=40, H=27.

Run the recorded experiment suite:

```
python -m src.seekaudio ablate
```

Then inspect `comparison.csv` and per-track reports. Keep new experiments in a separate output directory.

Write a short paragraph answering each question:

Why might MFCC-only beat the larger combined representation?

Does G versus E isolate PCA from whitening?

Does the experiment called “tuned” prove optimized weights?

Does 99.07% PCA variance retention imply almost perfect search?

What independent evidence would establish a real improvement?

Checkpoint

Propose one experiment that changes only one factor. Specify what stays fixed, how settings are selected, and which score will judge it. Do not select a winning setting on the final test set and then report that score as unbiased.

---

15. Stage 12: understand the interface and deployment

Learn

Streamlit reruns, widgets, session state, data/resource caches, content hashing, temporary files, saved-artifact paths, dependency pinning, and deployment branches.

Practice

Trace `streamlit_app.main`:

Load configuration and cached catalog.

Select upload/sample.

Hash selected bytes and clear stale analysis.

Preview the full recording.

On button click, analyze.

Store the result dictionary.

Display genre, plot, and technical values.

Change the upload name while keeping identical content. Then change content while keeping the same name. Observe why content hashing is preferable to filename-only identity.

Explain why a cloud model folder must include both vectors and fitted transforms. Demonstrate that the generated deployment folder works without `data/raw`.

Checkpoint

Explain why the webpage does not retrain models, why raw catalog paths need not exist on the deployed analysis-only app, and why a dependency update can require rebuilding or rechecking saved models.

Treat deployment as reproducible packaging, not as another opportunity to train secretly at startup.

---

16. Reconstruction milestones

Build a parallel exercise implementation in small files/notebooks. Do not overwrite the working baseline while learning.

Milestone	Deliverable	Evidence you understand it
1	File inventory and one metadata record	Correct fields and invalid-file handling
2	Mono/resampled centered clip	Predicted shape and duration
3	Frame matrix and spectrum	Correct axes/bin frequencies
4	RMS and spectral summaries	Controlled-tone/silence tests
5	Mel filter bank and MFCC record	Shape trace and plotted triangles
6	Rhythm record	Known pulse tempo and ambiguity explanation
7	Raw catalog table	Finite values and unique IDs
8	Fitted scaler/group/PCA pipeline	Query transform reuse
9	Cosine search	Known-vector ranking and self-exclusion
10	Evaluation reports	Hand-calculated metric agreement
11	Controlled ablations	One-factor experiment design
12	Upload analysis and deployment bundle	UI result and artifact consistency

A useful session format is 15 minutes of reading, 30-60 minutes of one experiment, and 10 minutes writing the result in your own words. Adjust the duration to your schedule; the purpose is regular evidence of understanding.

17. What to learn next for a stronger research project

High priority

Independent classification evaluation, artist/duplicate grouping, train/development/test protocol, confusion matrices, macro-F1, confidence intervals, open-set behavior, and error analysis.

The current app's genre vote needs its own benchmark. A retrieval score cannot substitute for it.

Medium priority

Robust scaling, carefully matched custom/library feature comparisons, multiple excerpt aggregation, temporal pooling alternatives, PCA-without-whitening, automatic weight search on development data, and model/data provenance hashes.

Later extensions

DTW for aligned feature sequences, cover identification, pretrained audio embeddings, learned metric spaces, approximate-nearest-neighbor search, and larger music datasets.

These are research directions, not required prerequisites to explain the current system. Do not list them as implemented until there is working code and evaluation.

18. Interview and admission discussion practice

Prepare a two-minute explanation:

> I built a content-based music retrieval and audio-analysis system. I represent recordings with pooled timbral, spectral, energy, and rhythm features, transform them into a comparable vector space, and retrieve neighbors with cosine similarity. I compared eight representations using same-genre catalog relevance and added a demo with a neighbor-vote genre estimate and a Mel spectrogram. The current results are catalog measurements; independent generalization and perceptual validation remain future work.

Be ready to answer:

Why 22,050 Hz and 30 seconds?

What information does mono conversion discard?

What does window size trade off?

Why Mel filters and a DCT?

How are MFCCs different from chroma?

Why are magnitude and power weighting different?

Why normalize groups after scaling columns?

How do weights affect cosine similarity?

What does whitening do?

Why does MFCC-only currently outperform combined features?

How do you transform an uploaded song consistently?

Which Librosa values are display-only?

Why is the genre output not calibrated?

What makes GTZAN evaluation difficult?

What would you do differently with independent test data?

Which parts did you implement directly and which libraries did you use?

Say “I haven't measured that yet” when appropriate. A reasoned limitation is stronger than an invented certainty.

---

19. A practical mastery checklist

You are comfortable with this project when you can:

Trace a file from waveform to feature dictionary to transformed vector.

Predict the default shapes without running code.

Derive RMS, centroid, bandwidth, rolloff, ZCR, cosine, and basic retrieval metrics.

Explain the MFCC pipeline and the custom HTK choices.

Explain both rhythm estimators and their ambiguities.

Explain optional tonal features without claiming transcription.

Reproduce one known-vector ranking and one metric example manually.

Transform a query with saved objects and explain why refitting is wrong.

Distinguish original metadata, excerpt statistics, retrieval features, and display-only features.

State the observed results and their evidence paths.

Design an independent evaluation with sensible grouping.

Diagnose a stale UI result, missing artifact, or incompatible dependency.

Explain the project clearly without reading its comments aloud.

---

20. Reading resources and order

Use official references and worked teaching material as supplements to your own experiments:

Python 3.12 tutorial for syntax and program structure.

NumPy beginner guide for array reasoning.

FMP notebooks for music-processing concepts and Python examples.

FMP STFT explanation for the time-frequency step.

The technical report's linked SciPy, scikit-learn, and Librosa APIs for the exact functions you use.

GTZAN analysis resources before designing stronger evaluations.

Read a method, predict an output, run a tiny experiment, and then return to the reference. Reading documentation alone does not establish that your pipeline is correct.

---

21. Suggested personal research notebook template

For each experiment, record:

```
Question:
Why it matters:
Baseline commit and feature version:
Dataset/candidate/query definitions:
Single changed factor:
Settings selected before test:
Expected observation:
Actual numerical result:
Listening or visual observation:
Possible confounders:
What the result does not prove:
Next experiment:
```

For a bug investigation, add **minimal reproducer**, **array shapes**, **dependency versions**, and **whether the issue is numerical, environmental, or a modeling assumption**.

This turns your project from an accumulation of functions into a technical learning record that you can explain and defend.

seekaudio: method-by-method reference

This atlas maps the production Python code to the technical report. Use the learning guide to study the prerequisites. Functions are listed by module, with their inputs, outputs, responsibilities, and important side effects.

The scope is the application, feature extractors, CLI, configuration classes, and deployment preparation script: **96 functions/classes**. Test functions are not production methods; the test suite is discussed in the technical report.

---

1. Start with the three call flows

Offline catalog preparation

```
__main__.main
  -> create_parser / load_config / execute_command
  -> cli_commands.run_all
  -> pipeline.run_pipeline
       1. ingest_dataset -> build_manifest -> create_manifest_record
       2. extract_dataset -> extract_feature_table
            -> extract_track_features -> load_audio
            -> extract_waveform_features -> individual extractors
       3. build_configured_space
            -> select_features -> build_feature_space
            -> standardize_features -> normalize_feature_groups
            -> apply_group_weights -> optional reduce_features_pca
            -> optional normalize_vectors
            -> attach_extraction_settings -> save_feature_space
       4. evaluate_precision_at_k -> find_similar_tracks
            -> calculate_ranking_metrics
            -> add_random_baseline -> save_evaluation
       5. run_ablations -> independent space fitting and evaluation
            -> plot_evaluation
```

External audio analysis

```
streamlit_app.main
  -> load_config / load_catalog
  -> select upload or sample / update_upload_state
  -> Analyze button -> analyze_upload
       -> analysis.analyze_audio
            -> read_audio_metadata
            -> load_audio
            -> extract_waveform_features
            -> search_feature_record
                 -> transform_features (NO FITTING)
                 -> find_similar_tracks
```

```
            -> add_track_details -> estimate_genre
            -> extract_librosa_characteristics
            -> render_mel_spectrogram
  -> show_analysis_result
       -> show_audio_characteristics / show_spectrogram
```

The bundled sample passes through the same analysis code using its existing local path. Uploads are written temporarily to supply a file path to the decoder and removed afterwards.

CLI search

```
search TRACK_ID
  -> run_search -> find_similar_tracks -> print_search_results

search-file FILE
  -> run_audio_search -> find_existing_track_id
  -> search_audio_file -> extract_track_features
  -> search_feature_record -> transform_features -> find_similar_tracks
  -> print_search_results
```

Genre labels are attached **after** acoustic ranking. They are not inputs to cosine similarity or PCA.

---

2. Entry point: main.py

Method	Input → output	Responsibility
`create_parser`	No input → `ArgumentParser`	Defines commands, option names, defaults, and help. Describing an argument does not execute a pipeline stage.
`execute_command`	Parsed arguments, configuration → `None`	Dispatches the selected command to its handler. Keeps command selection separate from implementation.
`main`	Optional list of command arguments → `None`	Parses arguments, loads configuration, dispatches, and handles expected command errors. The module entry guard calls it when run with `python -m`.

Learn here: imports, functions, command-line parsing, `None`, exceptions, the entry-point guard.

---

3. Command handlers: cli_commands.py

Method	Input → output	Responsibility
`get_match_count`	Arguments, configuration → integer K	Uses the explicit command K when supplied, otherwise the configured K. Validation occurs downstream.
`print_search_results`	Ranked rows, manifest → `None`	Adds track details and prints a readable search table. This changes presentation, not rankings.

Method	Input → output	Responsibility
`run_ingest`	Arguments, configuration → `None`	Resolves audio/output paths, creates the manifest, and prints the saved track count.
`run_extract`	Arguments, configuration → `None`	Loads the manifest and extracts the configured raw feature table with its JSON recipe.
`run_build_space`	Arguments, configuration → `None`	Loads raw features, applies CLI representation overrides, builds the space, attaches extraction settings, and saves paired artifacts. A components value of zero selects no PCA.
`run_search`	Arguments, configuration → `None`	Loads saved catalog vectors and metadata, searches by an existing ID, and prints results.
`find_existing_track_id`	Audio path, manifest → ID or `None`	Compares resolved file paths to identify a catalog-file query. It does not compare audio contents or recognize copied recordings.
`run_audio_search`	Arguments, configuration → `None`	Loads vectors/models/manifest, identifies an exact existing path for optional self-exclusion, extracts and searches the file, and prints matches.
`run_evaluate`	Arguments, configuration → `None`	Loads saved artifacts, evaluates all catalog queries, adds random expectations, and writes reports.
`run_comparison`	Arguments, configuration → `None`	Routes comparison commands to the earlier group comparison or the YAML ablation workflow.
`run_all`	Arguments, configuration → `None`	Calls the offline pipeline with configuration and ablation paths.

Side effects: printing, loading artifacts, and saving outputs for commands that build/evaluate. Search commands do not refit models.

---

4. Configuration: config.py

Configuration dataclasses

A dataclass is a convenient Python container with named fields. It is not an ML model.

Class	What it contains	Why it exists
`AudioConfig`	Sample rate, excerpt duration, mono flag	Describes waveform preparation.
`FeatureGroupsConfig`	MFCC/spectral/rhythm/energy/tonal switches	Selects columns used by the search representation.

Class	What it contains	Why it exists
`FeatureWeightsConfig`	One numeric weight for each group	Controls post-group-normalization multipliers.
`FeaturesConfig`	FFT/hop sizes, MFCC/Mel/chroma counts, nested groups/weights	Keeps extraction and feature-selection settings together. Nested factories avoid shared mutable objects.
`PoolingConfig`	`use_std`	Controls whether standard-deviation columns are selected; extraction still computes them.
`SpaceConfig`	PCA component count, whitening, final L2 switch	Configures vector-space construction.
`RetrievalConfig`	Default K	Sets search/evaluation neighbor count.
`PathsConfig`	Audio, manifest, feature, space, evaluation paths and feature version	Defines artifact locations and versioned raw-feature filenames.
`SeekAudioConfig`	All nested configuration objects	Gives methods readable access such as `config.audio.sample_rate`.

Configuration methods

Method	Input → output	Responsibility
`read_yaml`	Path → dictionary	Reads UTF-8 YAML with `safe_load`; an empty document becomes an empty dictionary.
`_deep_merge`	Base dictionary, override dictionary → merged dictionary	Recursively merges nested values without changing the original base. Scalars/lists replace earlier values.
`_dict_to_config`	Nested settings dictionary → `SeekAudioConfig`	Converts dictionaries to dataclasses, retaining defaults for unspecified fields. Copies feature settings before removing nested groups/weights.
`load_config`	YAML path, optional overrides → configuration	Checks file existence, reads settings, optionally merges overrides, and constructs named settings.
`load_ablation_configs`	Default and ablation YAML paths → name-to-configuration dictionary	Builds an independent complete configuration for each named experiment.

An underscore prefix indicates an internal helper by convention; it is not a security boundary.

---

5. Ingestion: ingest.py

Method	Input → output	Responsibility
`find_audio_files`	Root directory → sorted list of paths	Recursively discovers supported extensions and checks the dataset directory. Sorting stabilizes track order.
`read_audio_metadata`	Audio path → metadata dictionary	Reads original sample rate, channel count, sample-frame count, and duration from SoundFile. Does not compute MFCCs.

Method	Input → output	Responsibility
`create_manifest_record`	Audio path → manifest record	Combines metadata with filename-stem ID, parent-directory genre, and path.
`build_manifest`	Dataset directory → DataFrame	Reads records, warns/skips unreadable files, and validates usable track identities.
`save_manifest`	Manifest, destination → destination Path	Creates the parent folder and saves Parquet without a pandas index column.
`ingest_dataset`	Dataset and output paths → DataFrame	Coordinates manifest construction and persistence.

Distinction: original metadata describes the source recording; prepared-waveform measurements describe the normalized excerpt.

---

6. Offline orchestration: pipeline.py

Method	Input → output	Responsibility
`write_json`	Path, dictionary → `None`	Writes indented UTF-8 JSON. The parent folder must already exist.
`feature_path`	Configuration → Path	Constructs `features_v2.parquet` or another configured feature-version filename.
`extraction_settings`	Configuration → extractor-argument dictionary	Validates mono and 12-bin chroma support. Always enables rhythm extraction for later ablations; tonal extraction follows configuration.
`extract_dataset`	Manifest, configuration, destination → feature DataFrame	Rejects an empty manifest, extracts each track, saves Parquet, and writes the adjacent extraction-recipe JSON.
`build_configured_space`	Raw table, configuration → (vectors, models)	Selects groups/pooling, calls the fitting pipeline with weights/PCA/L2 settings, and attaches the complete configuration to models.
`attach_extraction_settings`	Model dictionary, feature-file path → `None`	Mutates models by attaching recipe/version from adjacent JSON when it exists. Does not guess settings for legacy files.
`run_ablations`	Raw features, manifest, config paths, output, K → comparison DataFrame	Fits each experiment independently on the same existing feature table, scores retrieval, saves reports/configurations, and orders summaries by precision. No audio re-decoding.
`run_pipeline`	Configuration and config paths → per-track evaluation DataFrame	Executes ingestion, extraction, representation fitting, evaluation, and ablations/plots. Writes outputs but only reads raw recordings.

Important: rerunning a build replaces artifacts at the configured output paths. Changing feature preparation requires rebuilding compatible downstream artifacts.

7. Waveform preparation: features/audio.py

Method	Input → output	Responsibility
`load_audio`	Path, target rate/duration, mono flag → waveform array	Decodes, validates, converts channels when requested, rationally resamples, and center-crops or symmetrically pads. Normal pipeline output is a mono float32 vector of 661,500 samples.
`split_into_frames`	Waveform, frame length, hop → frame-first matrix	Creates complete overlapping frames. Short inputs are padded enough for one frame; the ordinary tail beyond the final complete frame is not another partial frame. Default excerpt yields (1288, 2048).
`calculate_power_spectrum`	Frame matrix → power matrix	Applies a symmetric Hann window, computes one-sided real FFT, and squares magnitudes. Default shape is (1288, 1025).

See report Sections 6–7 for resampling, Nyquist, frame-count, Hann, FFT, and power equations.

8. Feature coordination: features/extract.py

Method	Input → output	Responsibility
`extract_track_features`	Audio path and extraction settings → dictionary	Loads the prepared waveform and calls the waveform-level coordinator. ID initially comes from filename stem.
`extract_waveform_features`	Prepared mono waveform, ID, settings → dictionary	Combines MFCC, spectral, and energy dictionaries, plus optional rhythm/tonal. Supports waveform reuse in live analysis.
`extract_feature_table`	Manifest, settings, progress switch → DataFrame	Iterates paths, extracts records, restores the manifest's authoritative ID, and optionally prints progress every 100 tracks. Errors propagate rather than silently dropping extracted rows.
`save_feature_table`	Feature table, destination → Path	Creates directories and persists raw numeric measurements plus ID as Parquet.

Default record: `track_id` plus **53** values. Default table: **999 × 54**. Extraction does not use genre labels to calculate the measurements.

9. Individual feature modules

features/energy.py

Method	Input → output	Responsibility
`extract_energy_features`	Waveform, frame/hop settings → two-value dictionary	Computes frame RMS from unwindowed samples and pools its mean/population standard deviation.

features/spectral.py

Method	Input → output	Responsibility
`extract_spectral_features`	Waveform, rate/frame/hop settings → eight-value dictionary	Pools mean/std of magnitude-weighted centroid, bandwidth, 85% magnitude rolloff, and frame zero-crossing rate. Handles silent spectral denominators.

features/mfcc.py

Method	Input → output	Responsibility
`_create_mel_filter_bank`	Rate, FFT size, Mel count → filter matrix	Places triangular filters uniformly in HTK Mel coordinates from zero to Nyquist. Default matrix: (128, 1025). No Slaney area normalization.
`calculate_mel_power`	Waveform and spectrum/filter settings → Mel-power matrix	Frames, computes power, and multiplies by the Mel bank. Matrix is frame-first: (1288, 128) by default.
`extract_mfcc_features`	Waveform and MFCC settings → 40-value dictionary	Converts Mel power to logarithmic dB values with a floor/range clipping, takes orthonormal DCT-II, keeps 20 coefficients, and pools mean/std.

Naming trap: `mfcc_01` corresponds to mathematical coefficient index zero, not index one.

features/rhythm.py

Method	Input → output	Responsibility
`extract_rhythm_features`	Waveform and framing/rate settings → three-value dictionary	Forms positive RMS changes, smooths them, pools onset mean/std, and estimates tempo from autocorrelation within 40–200 BPM. Weak/nonperiodic signals can return zero tempo.

This is the stored retrieval tempo, not the UI's separate Librosa tempo.

features/tonal.py

Method	Input → output	Responsibility
`extract_tonal_features`	Waveform and framing/rate settings → 36-value dictionary	Aggregates FFT power into 12 rounded pitch classes, normalizes chroma, projects to six tonal-centroid axes, and pools mean/std.

Optional, disabled by default. Does not implement harmonic separation, instrument recognition, or a calibrated key detector.

features/librosa_analysis.py

Method	Input → output	Responsibility
`extract_librosa_characteristics`	Prepared waveform and analysis settings → three-value dictionary	Transposes shared power into frequency-first form, computes spectral-flux tempo, mean flatness, and mean spectral contrast. Adapts contrast octave count to Nyquist. All-silent spectrum returns zeros.

Boundary: display-only dictionary; these three values do not alter the existing retrieval feature schema, PCA, or genre vote.

features/spectrogram.py

Method	Input → output	Responsibility
`compute_mel_spectrogram`	Prepared waveform and Mel settings → dB image matrix	Reuses custom Mel power, transposes to (bands, frames), and produces peak-relative display dB with an 80 dB range.

Default image matrix: **128 × 1288**. Relative normalization means independently generated images do not preserve absolute recording-level differences.

---

10. Representation fitting/inference: space.py

Method	Input → output	Responsibility
`get_numeric_features`	Feature table → numeric-column DataFrame	Requires ID, removes it from calculations, and rejects nonfinite values.
`create_feature_table`	Values, output column names, source table → DataFrame	Wraps transformed values while preserving source index and IDs.
`normalize_rows`	Numeric matrix → numeric matrix	Divides each row by Euclidean norm. Zero norms are replaced by one, preserving zero rows.
`get_group_columns`	Table and group name → column-name list	Uses prefixes such as `mfcc_` to locate a group.
`standardize_features`	Catalog feature table → (standardized table, scaler)	Fits `StandardScaler` on catalog columns and returns the reusable scaler. Constant columns become zero.
`normalize_feature_groups`	Standardized table → table	Normalizes each nonzero group separately for each track and rejects unknown feature prefixes. Returns a copy.
`apply_group_weights`	Group-normalized table and weights → table	Checks recognized groups and finite nonnegative weights, then multiplies each group. Missing weights leave groups unchanged.
`reduce_features_pca`	Weighted table, components, whitening → (component table, PCA)	Fits full-SVD PCA. Limits dimensions to min(track count minus one, feature count) and rejects whitening negligible-variance components.

Method	Input → output	Responsibility
`normalize_vectors`	Numeric-feature/component table → table	Applies whole-vector L2 normalization, retaining IDs and zeros. Distinct from per-group normalization.
`build_feature_space`	Selected catalog features, representation options → (vectors, model dictionary)	Validates IDs and runs the complete fitting sequence. Saves scaler/PCA, weights, input/output column order, and L2 setting.
`save_feature_space`	Vectors, models, directory → Path	Saves `vectors.parquet` and `models.joblib`. Only load trusted serialized models.
`select_features`	Raw table, enabled groups, std switch → table	Selects group columns and optionally removes `_std` columns. Requires at least one available enabled group.
`transform_features`	Raw query rows, saved models → vector table	Reorders required columns and applies saved scaler, group processing, saved PCA, and saved final normalization. Extra columns are ignored; missing required columns fail. **Never fits.**

The fitting/inference split is one of the project's most important architectural ideas. A query standardized against itself would not share the catalog coordinate system.

Default output: **999 × 41**, including ID and 40 PCA components. Do not interchange vectors and models from different fitting runs.

---

11. Retrieval: retrieval.py

Method	Input → output	Responsibility
`find_similar_tracks`	Vector table, query ID, K → ranked ID/similarity table	Validates identities/numbers, rejects zero query direction, compares cosine against all rows, removes self and zero candidates, clips roundoff, and stably sorts descending. Returns up to K.
`add_track_details`	Ranked rows, manifest → enriched rows	Joins genre/path by ID after ranking, validating unique identities and complete matches.
`search_audio_file`	File, catalog vectors/models, K, optional exclusion ID → ranked rows	Requires saved extraction settings, extracts a record, and delegates to record search.
`search_feature_record`	Extracted dictionary, catalog artifacts, K, optional exclusion → ranked rows	Applies saved transforms, checks vector schema, creates an unused temporary query ID, appends a query to a copy, and reuses the common ranker.

Self-ID exclusion does not detect other copies of the same recording. Similarity is a signed score, not a confidence percentage.

12. Live analysis: analysis.py

Method	Input → output	Responsibility
`estimate_genre`	Labeled nearest matches → genre string	Chooses most votes, then greatest summed similarity for vote ties. Exact remaining ties follow grouped genre order. Rejects empty/missing labels.
`analyze_audio`	File, vectors, models, manifest → analysis dictionary	Reads original metadata, loads one prepared waveform, extracts compatible retrieval features, searches ten neighbors, votes genre, computes sample peak/display-only Librosa descriptors, and renders a spectrogram PNG.

Output contains original duration/rate/channels; excerpt-based technical measurements; analysis duration; genre; PNG bytes. A silent peak is `None` rather than a finite dBFS value. UI code handles formatting.

This method is not training a supervised classifier. The neighbor list is used internally even though it is hidden in the simplified interface.

13. Evaluation: evaluate.py

Method	Input → output	Responsibility
`calculate_ranking_metrics`	Ordered binary relevance array, total relevant candidates → metric dictionary	Computes Precision@K, Recall@K, AP@K, and nDCG@K. AP denominator is min(returned count, total relevant). Requires a nonempty ranked array.
`evaluate_precision_at_k`	Vectors, labeled manifest, K → per-track DataFrame	Queries every catalog ID, excludes itself, compares result labels with query genre, and computes metrics. Validates K, labels, unique IDs, and nonzero vectors.
`add_random_baseline`	Fully evaluated per-track results → copied DataFrame	Adds exact expected random precision: (genre count minus one)/(catalog count minus one). No Monte Carlo sampling.
`save_evaluation`	Results, K, output directory → Path	Writes per-track CSV, per-genre CSV, and summary JSON. Saves optional metrics/baselines when supplied and computes lift when random expectation is positive.

The method `evaluate_precision_at_k` retains its original name but now calculates four metrics. Mean AP over all query rows is reported as MAP@K.

14. Earlier comparisons: experiments.py

Method	Input → output	Responsibility
`compare_feature_groups`	Raw features, manifest, K, output → comparison DataFrame	Runs the earlier MFCC/spectral/energy group study, fits each representation, evaluates, and saves comparison outputs.

The main `run` workflow now uses the eight YAML ablations through `pipeline.run_ablations`. Do not confuse the earlier comparison command with that configured experiment set.

15. Visualization: viz.py

Method	Input → output	Responsibility
`plot_evaluation`	Per-track scores, ablation summary, directory, K → `None`	Saves horizontal genre/ablation precision bar charts. Genre chart includes random reference; axes span zero to one. Closes pyplot figures.
`render_mel_spectrogram`	Prepared waveform and settings → PNG bytes	Builds custom log-Mel data, positions frame/filter pixel centers, labels time and Mel-spaced Hz ticks, and renders a dark-theme image into memory using headless Matplotlib.

The image time axis refers to the excerpt, not its offset in the full file. Color is peak-relative dB, not dBFS or calibrated sound-pressure level.

16. Web interface: streamlit_app.py

Method	Input → output	Responsibility
`read_table`	Path string, modification timestamp → DataFrame	Cached Parquet loading via `cache_data`. Timestamp is part of the cache key even though the function body only uses the path.
`read_models`	Path string, modification timestamp → model dictionary	Cached trusted Joblib loading via `cache_resource`. Timestamp invalidates changed artifacts.
`load_catalog`	Configuration → (manifest, vectors, models)	Resolves paths relative to the app project root and loads the prebuilt catalog with modification-keyed caching.
`configure_page`	No input → `None`	Sets page configuration and CSS, including hiding the default top toolbar. Does not configure feature extraction.
`update_upload_state`	Audio bytes → `None`	Hashes selected content with an analysis-version prefix and resets stale session results when the selected audio changes.
`analyze_upload`	Upload, manifest, vectors, models → dictionary	Writes a decoder-readable temporary file, calls analysis, and guarantees cleanup.
`show_spectrogram`	Analysis dictionary → `None`	Displays PNG, caption, and download control.
`show_metric_row`	Measurement specifications → `None`	Creates columns and shows one row of formatted metrics/help text.
`show_audio_characteristics`	Analysis dictionary → `None`	Formats fourteen measurements and displays them in rows, explaining excerpt-based analysis.
`show_analysis_result`	Analysis dictionary → `None`	Shows genre estimate with its limitation, characteristics, and spectrogram.

Method	Input → output	Responsibility
`main`	No input → `None`	Loads catalog, offers upload/sample, previews audio, handles Analyze-button spinner/errors, stores results in session state, and displays existing results on reruns.

Two different forms of reuse:

Shared file/model caches avoid unnecessary artifact reloads.

Session state keeps a particular visitor's selected audio and analysis result across Streamlit reruns.

Neither means new uploads are added to the catalog or used for model training.

---

17. Deployment preparation: scripts/prepare_streamlit.py

Method	Input → output	Responsibility
`prepare_deployment`	No arguments → deployment-directory Path	Copies required source/config/dependencies and saved manifest/vectors/models to a small bundle, including the optional sample when present. Checks required inputs before copying.

This prepares local files, not a hosted deployment. It does not push Git, choose a cloud branch, or remove the original repository's history. It leaves unrelated existing bundle files alone; rerunning can overwrite matching destination files.

---

18. Reading a function independently

For any method, answer these six questions in your notebook:

Input: Is this a file path, waveform, frame matrix, dictionary, or DataFrame?

Shape/units: What does each axis or column represent?

Computation: Is it reading, calculating, fitting, transforming, ranking, or displaying?

Output: What shape/type and units leave the method?

Side effects: Does it write disk, print, mutate a dictionary, or update UI/session state?

Failure cases: What happens for silence, missing columns, duplicate IDs, invalid settings, or unreadable audio?

Do not treat all `dict` values as equivalent: an extraction record, a model dictionary, and a display-analysis dictionary have different contracts.

19. Practical navigation exercises

Exercise A: follow a catalog recording

Choose one manifest row. Locate its path and genre. Follow `extract_track_features`, count the 53 numeric fields, then follow `build_feature_space` to the 40 final components. Explain why genre is absent from the transformation.

Exercise B: follow an upload

Set breakpoints at `analyze_audio`, `extract_waveform_features`, `transform_features`, and `estimate_genre`. Inspect shape, dtype, field names, and output type. Verify there is no `fit` call on this path.

Exercise C: explain one wrong-looking result

If tempo appears half/double what you expect, trace the separate stored and displayed rhythm methods. Compare periodicity and listen. Avoid changing PCA models merely to adjust a display value.

Exercise D: reproduce one evaluation row

Inspect ten ranked neighbors for a query. Create the binary genre-relevance sequence and manually calculate four ranking metrics. Compare with `calculate_ranking_metrics` and the saved CSV.

Exercise E: explain an artifact mismatch

Describe why a vector file from one fitting run and a scaler/PCA dictionary from another may have identical column names yet incompatible coordinates. Explain why artifacts should be generated, stored, and deployed as a matched set.

Return to the technical report for formulas and the learning guide for the reconstruction sequence.